CRYPTALYST

Design and Architecture Reference

Edition 2 · envelope · correlation · data lake · orchestrator · security · ticker

Field	Value
Document	CryptAlyst Design and Architecture Reference
Edition	2 — supersedes the platform reference for architecture decisions
Status	living document; parts I to III and V normative, part IV current measurements
Audience	engineers, quants, risk, operations, and anyone asked to change the platform
Scope	the running paper-trading platform and the ATLAS design parts adopted into it
Companion	CryptAlyst-Platform-Reference.docx (model mathematics and defect ledger)

Reading order. Part I explains what the platform is and the principles it is held to. Part II is the architecture itself. Part III is sixteen use cases — each one a need, the ATLAS mechanism that answers it, how this platform delivers it, and the criterion that proves it. Part IV is the measured evidence, the delivery plan and the decisions still open. Part V chooses the storage, and specifies the security and ticker services in the same style.

Contents

Part I — Context

1 Executive summary

2 Principles the platform is held to

3 The platform as it stands

4 Goals, non-goals and constraints

Part II — Architecture

5 Four planes and one bus

6 The envelope

7 The correlation service

8 The agnostic data lake

9 The orchestrator

10 Bus topology, priority and quality of service

11 Implementation map

Part III — Use cases

12 Sixteen use cases, from market data to compliance

Part IV — Delivery

13 Measured performance

14 Test and verification strategy

15 Rollout plan and acceptance gates

16 Risks, open decisions and the invariant

Part V — Storage, security and the ticker service

18 Storage selection and sizing

19 The security service

20 The ticker service

21 Appendix — topics, glossary, file map

Appendix A — Database technology considerations

Appendix B — Compliance

Appendix C — Trade replay

Part I — Context

1 Executive summary

CryptAlyst is a paper-trading platform for crypto markets. A Go market-data service streams 161 websocket channels from the exchange; a RabbitMQ bus carries nineteen topics; thirty Python model books trade against it; a dashboard shows what happened. It works, and until this edition it had no shared object describing what a message was, how fresh it was, what produced it, or which inputs it came from.

This edition adopts four parts of the ATLAS design and makes them normative:

- The envelope — one fixed shape carrying data, metadata and references, extensible in metadata only.
- The correlation service — minting is local, mapping is central, resolution is gated and recorded.
- The agnostic data lake — one API, tiered storage, retention with rotation and compression, replay from disk.
- The orchestrator — capabilities and generations instead of process liveness and fixed timers.

The claim of the document is not that these are good ideas. It is that each one removes a specific defect that the platform demonstrably has, and that the removal is measurable. Part IV contains the measurements: an envelope validates in 2.5 microseconds, the lake writes 400,000 envelopes per second and replays 1,127,000 per second, and the recorder has run for hours with zero errors.

1.1 What changed in this edition

Area	Before	After
Message shape	bare JSON per producer, no version, no provenance	envelope with version, plane, producer, generation, correlation id
Priority	claimed in comments	plane in metadata, separate topology, measured lag
Indicators	recomputed by every consumer	published once as advisories with references to inputs
History	fills written by each book in its own format	every envelope recorded once, in tiers, with retention
Backtest	REST candles, different code path	replay of recorded envelopes through the same calculators
Failure	log lines on disk	correlation chain: what happened, why, what was refused and on which rule

1.2 The central finding

The platform's problems were not model problems. They were lineage problems. When a trade lost money, no one could say which advisory caused it, which ticks produced that advisory, or whether the forecast was forty minutes old. That is why the first three defects in the previous reference kept recurring: they were symptoms of a missing abstraction, not independent bugs.

> **A trading platform without lineage cannot learn. It can only accumulate trades.**

2 Principles the platform is held to

Twelve principles come from ATLAS and are adopted unchanged. They are stated here in trading language because a principle that cannot be tested against a trade is decoration.

#	Principle	How it is tested here
P1	One contract	every message on every topic decodes as an envelope
P2	The shape never changes	adding a model or a calculator changes no consumer code
P3	Contents are somebody's job	the payload is validated per topic; metadata is extensible
P4	One identity	a correlation id follows a decision from tick to fill
P5	Refer, do not copy	an advisory points at the ticks it used; it does not embed them
P6	Authority before action	the risk interceptor refuses before the order is constructed
P7	Evidence is an output	the decision envelope records the refusal and the rule named
P8	Secure by construction	resolution is gated and every read is itself recorded
P9	No re-engineering	new capabilities register; nothing behind them changes
P10	Platform agnostic	storage backends are swappable behind one interface
P11	Honest metadata	age, generation and producer travel with the value
P12	Incremental forever	each phase leaves the platform running and measurable

2.1 Three trading-specific principles

The twelve above are general. Three more are specific to trading, and they are the ones that decide whether the platform is safe to leave running unattended.

Principle	Meaning	Why it is not negotiable
Market data first	the data plane is never blocked by the compute plane	a delayed price is a wrong price, and every decision downstream inherits the error
Refuse on evidence, not on suspicion	a refusal quotes the age, generation and rule	a refusal without evidence cannot be reviewed, and unreviewable refusals become folklore
Record what explains, not everything	advisories and decisions always; market data by policy	unbounded recording turns the recorder into the outage

3 The platform as it stands

The inventory below is the running system, not a plan. Ports and ownership matter because the migration is half complete by design: Go owns the data path, Python owns the books.

3.1 Processes

Process	Language	Port	Owns	Talks to
marketdata	Go	8796	exchange streams, prices, depth, candles	Binance websocket, bus, HTTP readers
busconsumer	Go	8798	one consumer per topic queue, tallies	bus
healthpub	Go	—	probes seven services, publishes system.health	bus
signalbus	Go	—	alert intake from the detector	backend API, bus
recorder	Go	8801	the lake: memory, journal, retention	bus, disk
replay	Go	—	re-emits a recorded lake	disk, bus
signals	Python	—	the 20-field signal set and flow confidence	market data, bus
tad	Go	8792	technical analysis per symbol	bus
forecast	Go	8790	the seven-agent blend	bus
supervisor	Python	—	fleet lifecycle for the books	processes, state
master	Python	8799	dashboard, 38 endpoints, SSE	everything
30 model books	Python	—	trading decisions and books	bus, risk, tactics

3.2 What the data path looks like

One process touches the exchange. Everything else reads either the bus or a local endpoint. This is the property that makes the Go migration safe: the feed can be replaced without the books noticing.

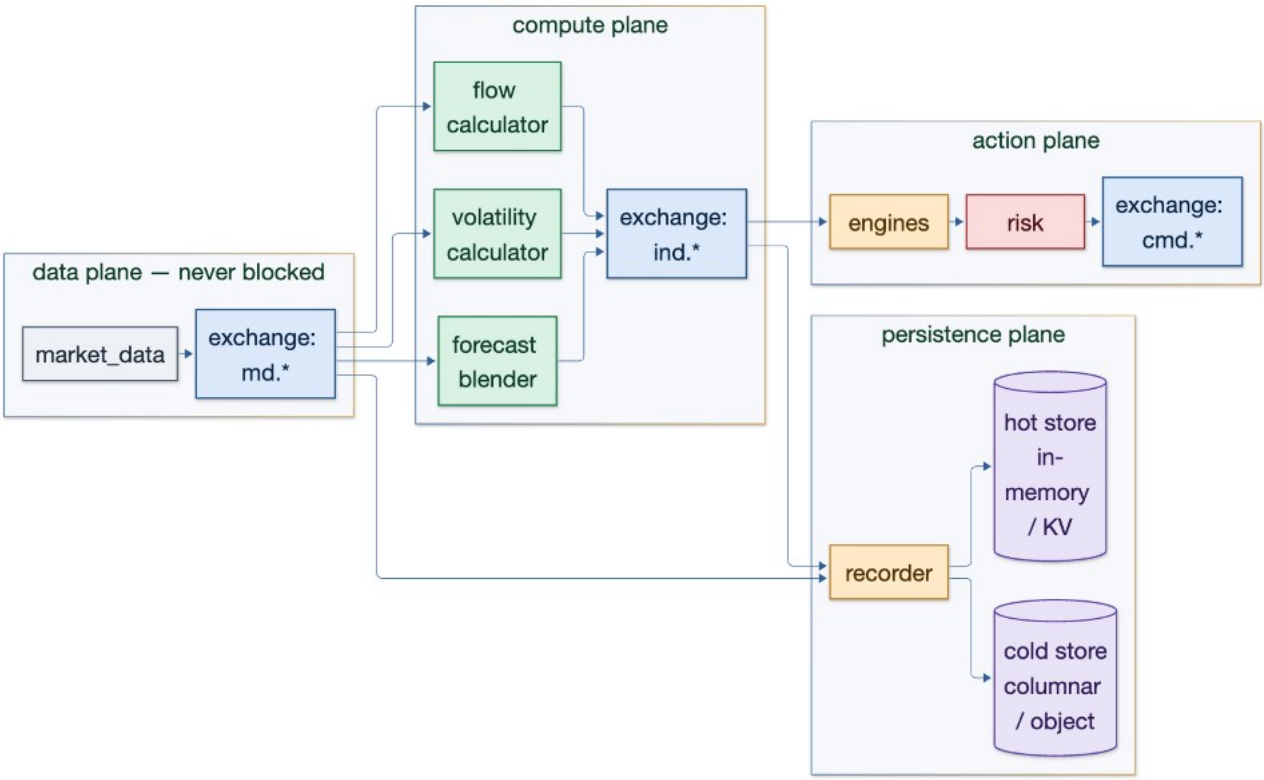


Figure 3.1 — Four planes. Market data has its own; persistence is a consumer, never inline.

3.3 What is not yet true

Gap	Consequence today	Addressed in
Python books do not emit envelopes	decisions are recorded but not linked to advisories	phase 5, section 15
Orchestrator not built	dispatch is still fixed-interval loops and a supervisor	phase 4, section 9
Retention not yet enforced on the live lake	the journal grows without a window	phase 2, section 8.4
Backtest harness still separate	replay exists but is not wired to the harness	phase 6, section 15

4 Goals, non-goals and constraints

4.1 Goals

Goal	Measure of success
Every message carries provenance	100% of recorded envelopes decode with a version, plane and producer
A decision can be explained	any decision resolves to advisory, ticks and candle in one query
Backtest equals live	replaying a window reproduces the same decisions
Market data is never blocked	market plane lag stays under one bar interval while compute saturates
Storage is bounded	lake size has a window per tier and compressed closed segments
Adding a model is a registration	no existing service changes when a model is added

4.2 Non-goals

- Real money execution. This platform is paper-only by design and by instruction.
- Multi-venue arbitrage. One venue, one feed, one truth; multi-venue is a later envelope plane.
- Sub-millisecond colocation. The envelope is microsecond-cheap, but the architecture targets seconds, not ticks.
- A general-purpose streaming platform. Every abstraction here exists to explain a trade.

4.3 Constraints

Constraint	Effect on the design
The books stay Python	risk, tactics and maker remain in-process libraries; the envelope crosses the bus, not every call
One broker (RabbitMQ)	priority is enforced by topology and pool sizing, not by separate infrastructure
A single operator	no service may require a second person to run or recover
Exchange rate limits	streams only; the boot ranking call is the single permitted REST request and it falls back

Part II — Architecture

5 Four planes and one bus

The platform is organised into four planes with different failure modes and different budgets. The bus is the only thing they share, and the envelope is the only contract on it.

Plane	Traffic	Failure mode	Policy
Market	ticks, depth, candles, 24h statistics	a dropped update means a stale price	publish first, drop oldest, never block
Compute	advisories, forecasts, TA, regime	a late advisory is usable if it says so	at least once, keyed by symbol
Action	decisions, fills, risk state, control	a lost decision is an audit hole	durable, ordered per symbol, never dropped
Persistence	everything, recorded once	a slow recorder must not slow the feed	consumer only, batched, bounded by retention

Two rules follow, and they are the whole of priority in this design:

- A publisher never fans out. One message per event; the broker copies it.
- Market data consumers are disposable. If one falls behind it is dropped and re-syncs; it never applies backpressure to the feed.

5.1 Why the bus is the spine

Every dependency in the platform is a subscription. That makes the graph inspectable: what a model needs is the set of topics it reads, and what it produces is the set it writes. A service that bypasses the bus is a service whose dependencies cannot be seen, which is why the legacy direct-call paths were removed.

6 The envelope

The envelope is a fixed shape: data, metadata, references. It is the only contract between the data layer and everything else. Because the shape is fixed, new requirements do not reopen old code — behaviour lives around the envelope in operators, interceptors and transformers.

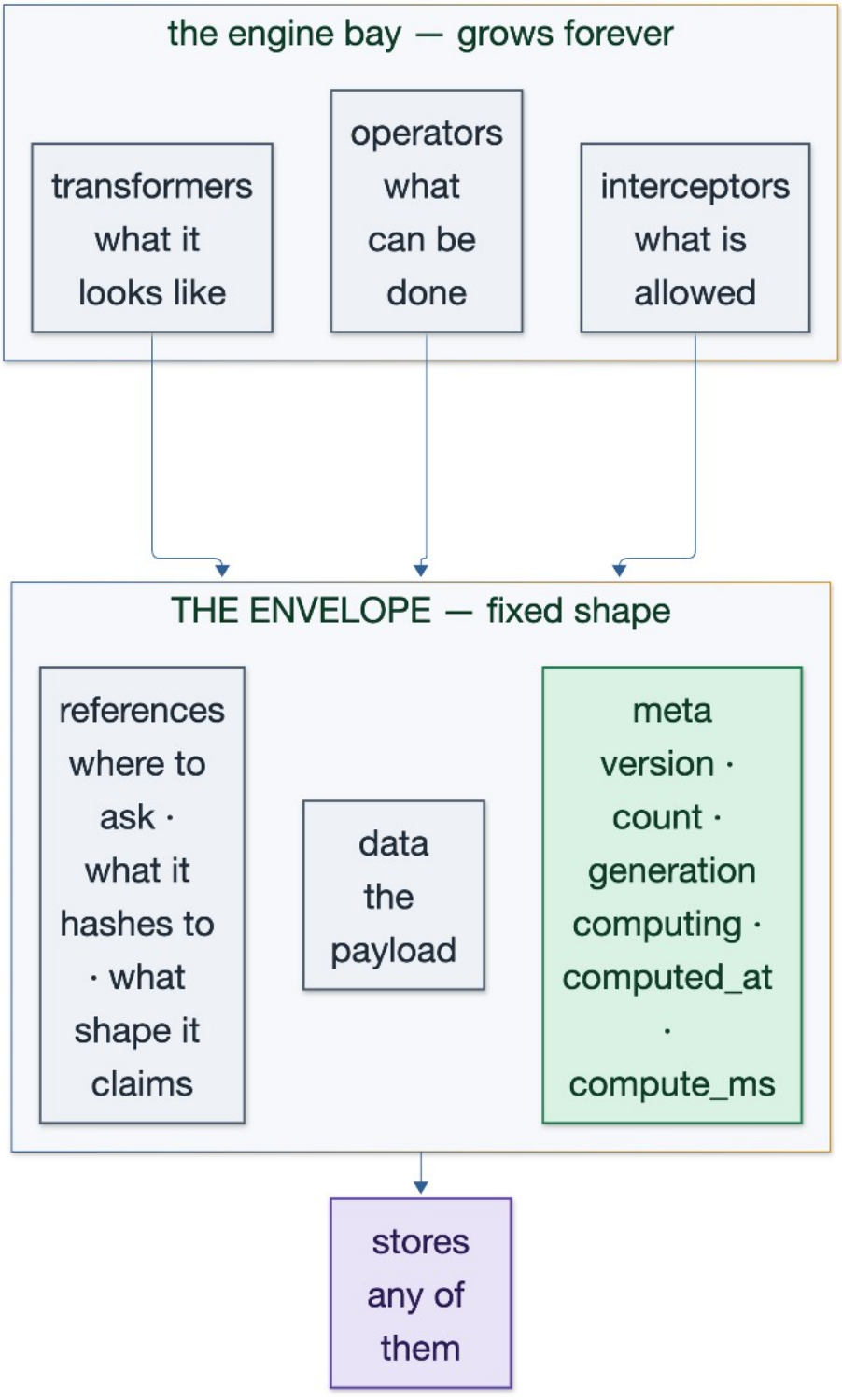


Figure 6.1 — The fixed shape, and the engine bay that grows around it.

6.1 The three members

Member	Carries	Never carries
data	the payload for this topic	storage types, driver handles, or a second copy of another topic's data

meta	version, plane, producer, generation, sequence, computed_at, received_at, compute_ms, cid, refs	anything a reader must parse to understand the payload
refs	topic, key, generation, cid of an input	the input itself

6.2 Versioning policy

A reader accepts the current revision and the one before it, refuses anything newer, and treats an absent version as revision one. The payload is strictly validated: an unknown top-level member is refused. The metadata is extensible: an unknown meta key is ignored. That asymmetry is deliberate — provenance can grow without breaking readers, while a malformed payload cannot silently become a trading decision.

```
AcceptVersion(0)       -> ok  (pre-versioning revision)
AcceptVersion(1)       -> ok  (current)
AcceptVersion(2)       -> refused  (newer than this build reads)
Decode({data,meta})    -> ok
Decode({data,meta,x}) -> refused  (unknown top-level member)
Decode(meta{x_new})    -> ok  (metadata is extensible)
```

6.3 The lifecycle: append, monitor, satisfy

An envelope is not a packet that dies on delivery. It travels, being monitored, until the minimum requirements are satisfied; work that can help appends a step and a correlation id.

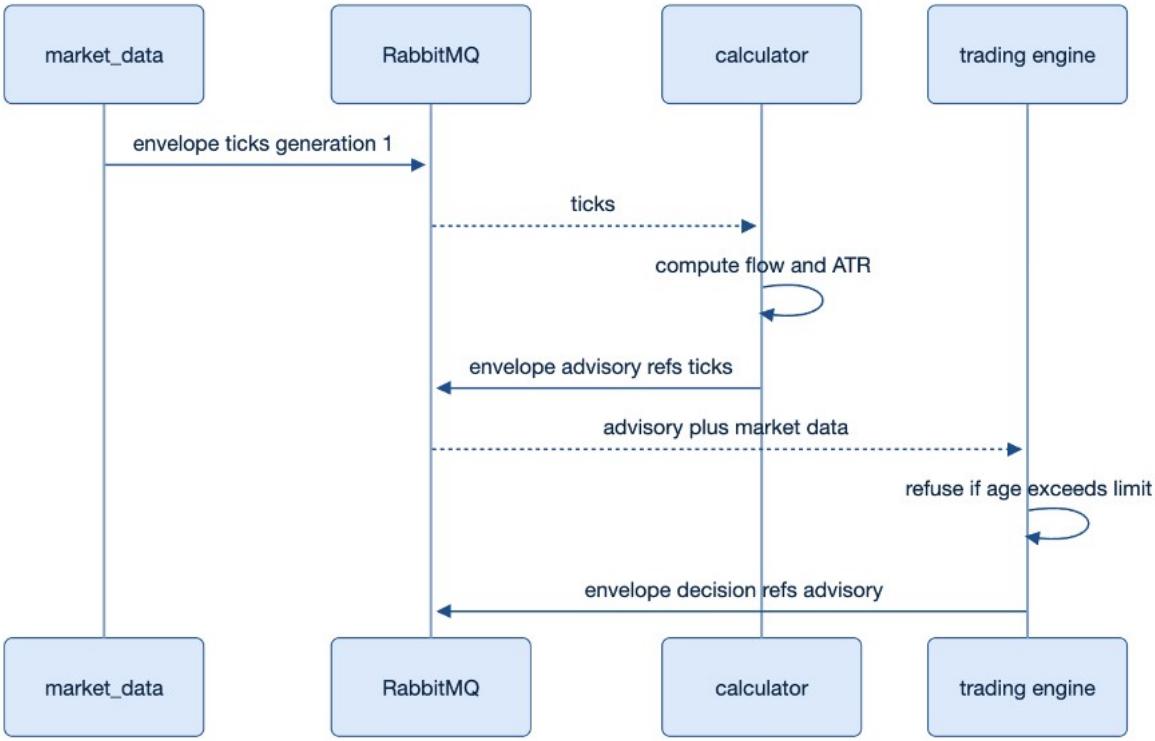


Figure 6.2 — One decision, three envelopes. Each stage appends provenance; none changes the shape.

7 The correlation service

The service maintains the correlation map: which document exists, which needs it carries, who promised what, which step fulfilled which need, and which actor may do what. It does three jobs — introduce, pair, resolve — and its design rests on one asymmetry.

```
minting is local      a cid is created at the boundary that created the work
mapping is central    someone must own who is doing this for whom
resolution is gated   reading the map is privileged, and every read is recorded
```

That asymmetry is why the service can be down without stopping trading. Work still starts, promises are still made and fulfilments still append; only introduction and resolution wait.

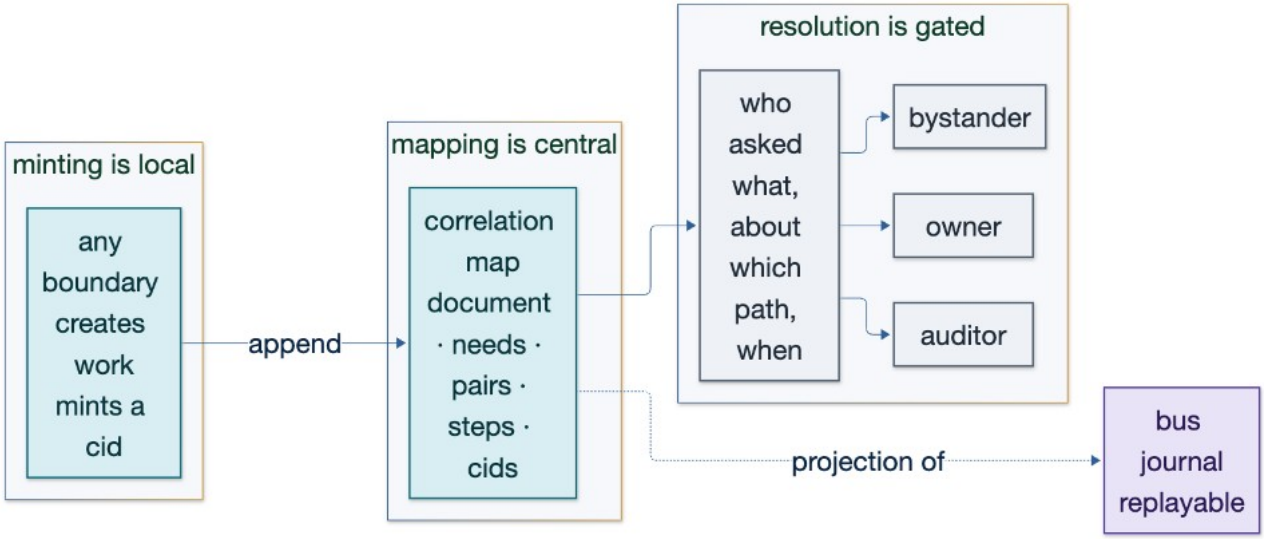


Figure 7.1 — Mint anywhere, map centrally, resolve under authority.

7.1 The map is a projection

The map is built from durable records — the journal, the ledger, the capability registry — and never holds the only copy. It can be rebuilt by replaying, which is what makes a restart a non-event and more than one instance possible.

7.2 Two rules that keep a trace trustworthy

- Every output of the service is either a step on an existing envelope or a new envelope. There is no private API that bypasses the contract.
- Resolution is recorded on the document being read, so reading the trace becomes part of the trace.

The second rule answers a question a desk will ask on day one: who looked at this decision, and when? A resolver without that property is an audit hole dressed as an audit tool.

8 The agnostic data lake

The lake solved, in ATLAS, exactly the problem CryptAlyst had: a single JSON file that grew until every read was a scan and every aggregate was recomputed. The fix was not a faster query. It was a tiered, sharded, pluggable store behind one envelope API.

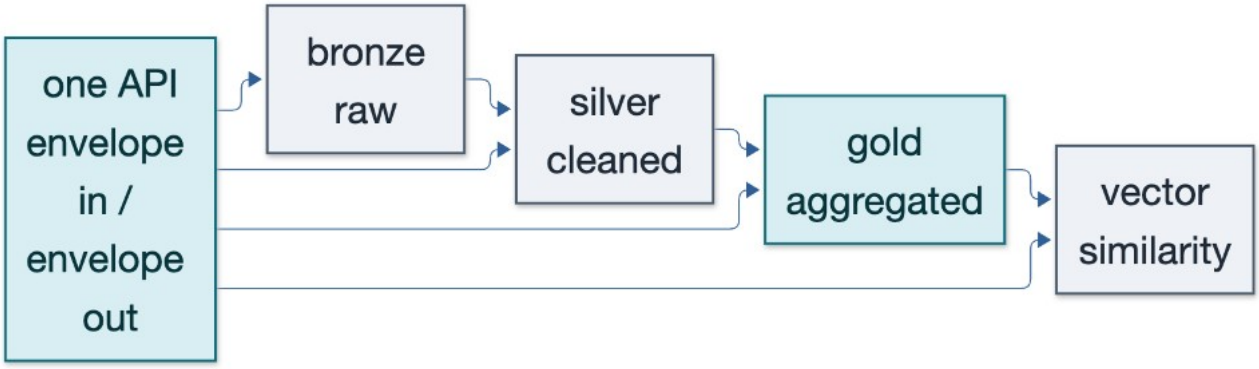


Figure 8.1 — One API, four tiers, each backend replaceable.

8.1 Tiers and what lives in each

Tier	Contents	CryptAlyst use	Retention
Bronze	raw envelopes exactly as received	replay and forensic reconstruction	hours to days
Silver	cleaned and normalised records	bars, flow, volatility for research	days to weeks
Gold	materialised aggregates	advisories, per-model performance, dashboards	weeks
Archive	compressed closed segments	regulatory and historical study	months

8.2 Backends are swappable

Every tier implements one interface: write a record, close. The consumer never learns which store answered, and the recorder fans one write out to all configured backends. Two are implemented today, both with tests.

Backend	Behaviour	Benchmark
memory	keeps the latest N per topic, drops oldest	1.85 microseconds per write
segmented-journal	rotates, compresses closed segments, deletes expired	2.39 microseconds per write
fan-out (both)	one call, two durable outcomes	2.41 microseconds per write

8.3 Retention is part of storage

Recording everything is only affordable if what is recorded has a lifetime, so the policy lives with the storage rather than in a cron job: rotate at a size, compress what was closed, delete what has aged out.

```
RetentionConfig{ SegmentSize: 64 MiB, MaxAge: 1h, Compress: true }
seg-<unixnano>.jsonl  -> closed -> seg-<...>.jsonl.gz -> older than MaxAge -> removed
```

Three tests prove it: rotation happens at the size boundary and replay across segments returns every record; closed segments are compressed and replay still works through the gzip; expired segments are deleted and counted. Measured compression on live traffic was 44.7% of original size.

8.4 Replay

Replay reads a journal back in order and re-emits it. Because the stored object is the envelope that travelled, replaying feeds consumers the same objects they saw live. This is the mechanism that makes a backtest a replay rather than a re-derivation.

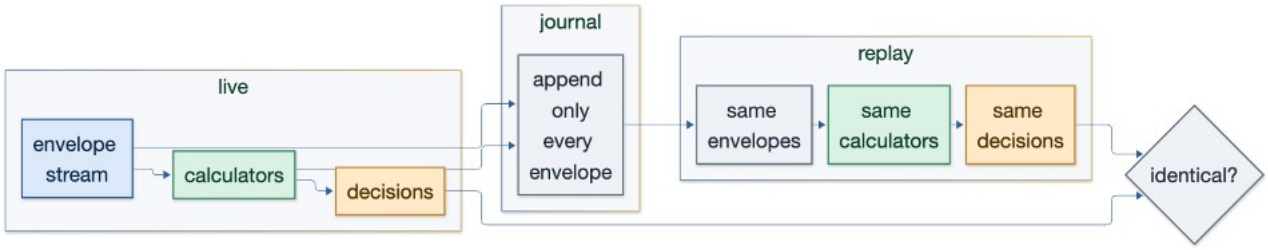


Figure 8.2 — Journal on the left, replay on the right, identical envelopes between them.

9 The orchestrator

A supervisor restarts processes on a timer. An orchestrator asks for a capability and receives an answer with metadata attached. The difference is between knowing a service is alive and knowing its answer is current.

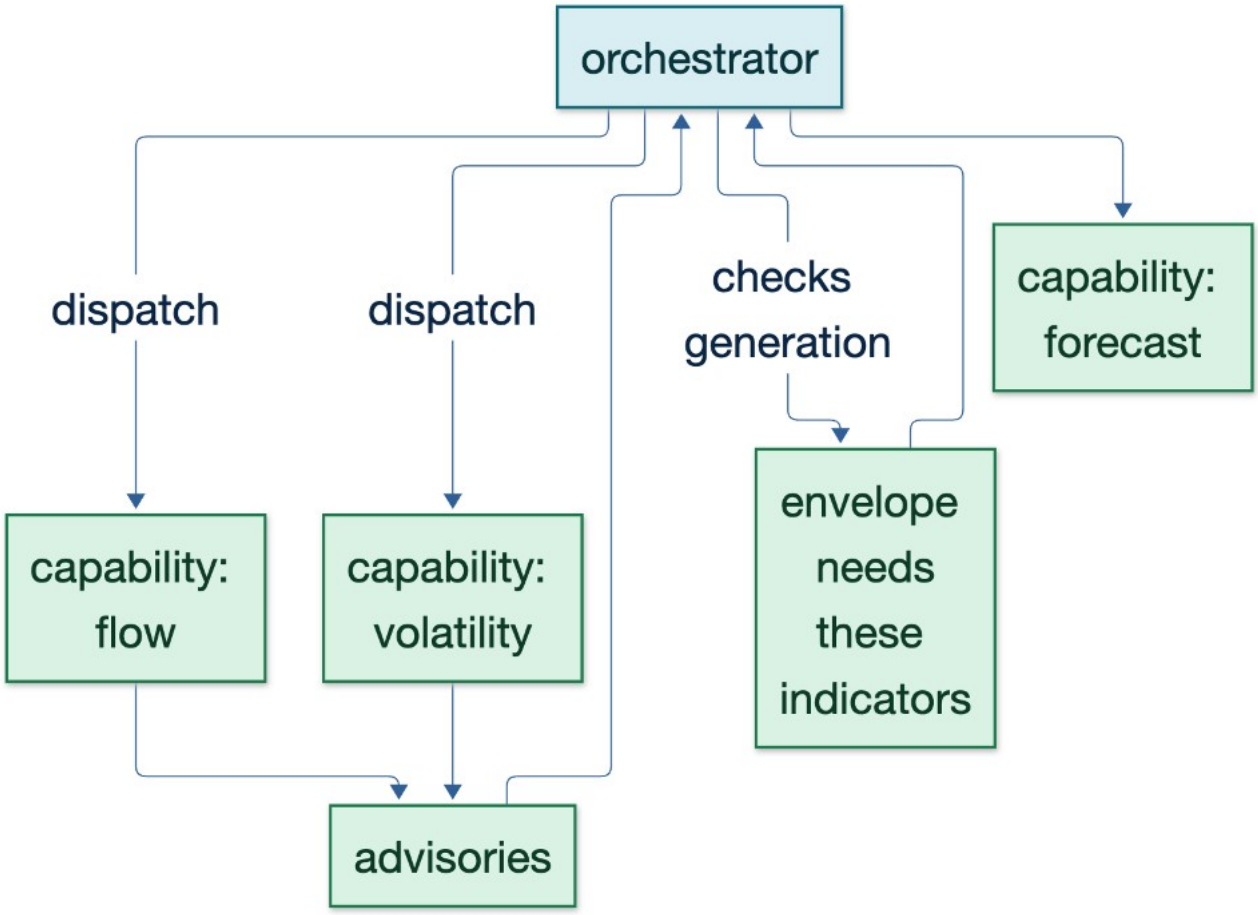


Figure 9.1 — The orchestrator never knows what a calculator computes; it knows what it can do and whether its answer is fresh.

9.1 Responsibilities

Responsibility	Today	With the orchestrator
What to compute	every model runs on its own clock	a capability is requested when an envelope needs it
When to recompute	fixed loop intervals	generation and computed_at decide whether the answer is current
What to restart	process liveness only	a consumer whose lag exceeds its contract is rescheduled
What to persist	books write their own JSON	the recorder owns persistence; models do not
What is allowed	risk checks inline in each model	an interceptor refuses before the work starts

9.2 Dispatch contract

A capability advertises a name, the topics it consumes, the topics it produces, a deadline, and a generation. The orchestrator matches an unsatisfied need to a capability, records the introduction as a step on the document, and expects a fulfilment envelope within the deadline. Nothing about the contract mentions a process, a port or a language.

The orchestrator's ignorance is the feature. It is the only component that can add a thirty-first model without being redeployed.

10 Bus topology, priority and quality of service

One broker, one topic space, and a policy per class. Priority is enforced by separating pools, not by trusting a flag.

Class	Topics	Delivery	Consumer policy	On overflow
market	ticks, market.price, market.depth, market.candle, market.mini24h	at most once	own pool, small prefetch	drop oldest
intel	intel.signal, intel.flow, intel.ta, intel.forecast, intel.movers	at least once	shared compute pool	queue, bounded
model	model.decision, model.fill, model.pnl, model.rank	at least once, keyed	durable	queue, never drop
risk/control	risk.state, exec.quote, alert.pumpdump	at least once	durable, high priority	never drop
system	system.health, review.verdict	at most once	best effort	drop oldest

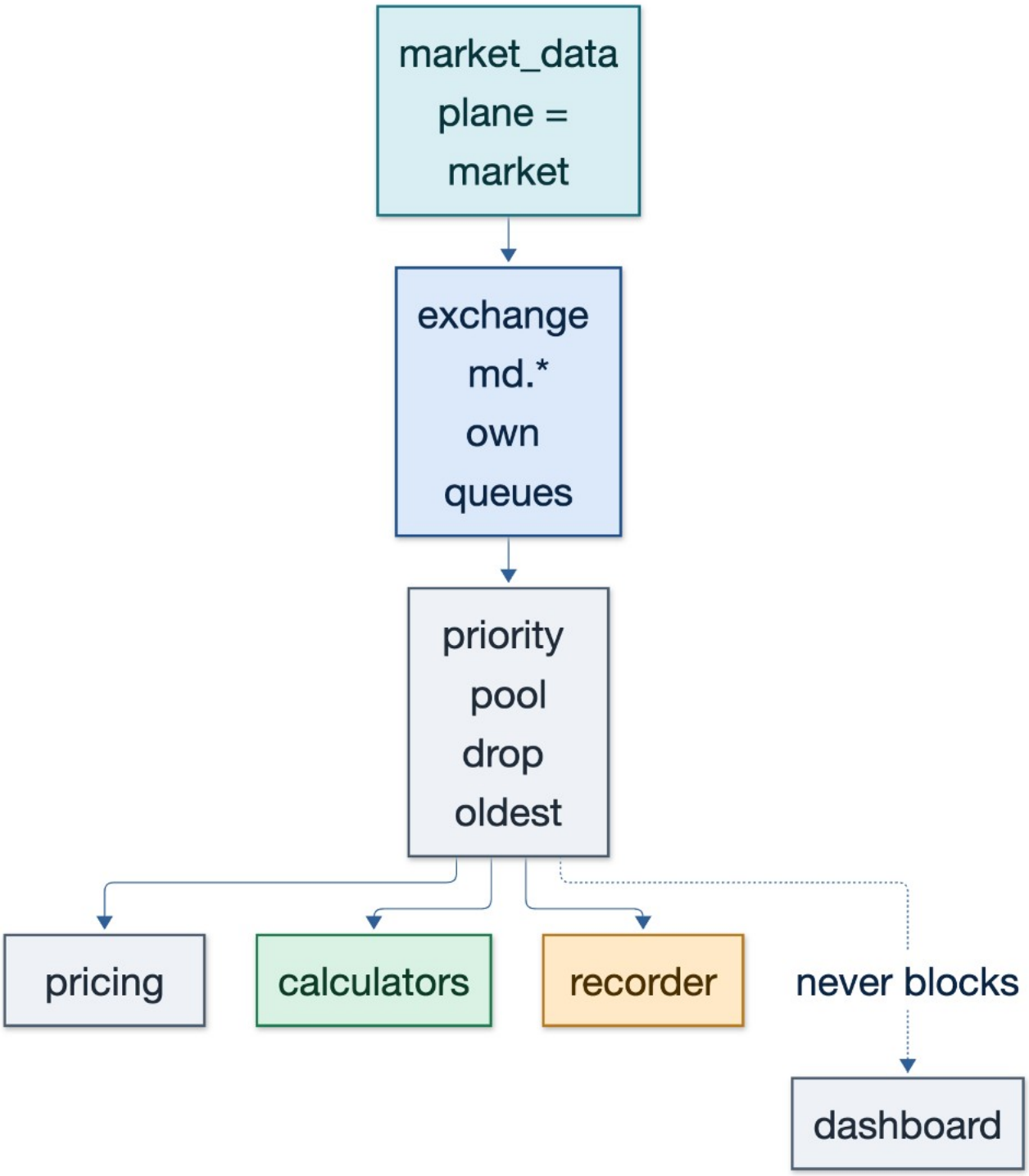


Figure 10.1 — The market-data plane has its own exchange, queue family and consumer budget.

10.1 What was measured after the changes

The priority claim is not asserted; it is observed. Two measurements matter: whether the market plane kept publishing while compute saturated, and whether payload sizes stayed bounded after the snapshot fix.

Measurement	Result
depth published per second	38.8 per second across 40 symbols, change-driven with a one-second heartbeat
ticks envelope size	7.4 KB after restricting the snapshot to streamed symbols (was 98.6 KB)
ticks messages carrying the full universe	0 — the filter is covered by a unit test
recorder errors over the run	0

11 Implementation map

Where each part of the design lives, in the repository, today.

Design part	Package	Tests	Status
envelope	core/internal/envelope	7	live
correlation service	core/internal/correlation	5	live
lake: memory, journal, retention	core/internal/lake	6	live
market data state and throttling	core/internal/marketdata	3	live
recorder	core/cmd/recorder	—	live on :8801
replay	core/cmd/replay	—	live
bus consumer, health, alerts	core/cmd	—	live
orchestrator	not built	—	designed in section 9, phase 4

The Python side keeps three libraries that the books call in-process — risk, tactics and maker — because putting a network hop in front of every trade decision would cost more than the coupling it removes. The envelope crosses the bus; these three remain part of the book layer.

Part III — Use cases

Sixteen use cases follow, each in the same four-column form: the need as it exists in this platform, the ATLAS mechanism that answers it, how CryptAlyst delivers that mechanism, and the criterion that proves delivery. They are ordered from the data plane outwards to research and compliance.

12 Sixteen use cases

UC-01 Market data stays first when compute saturates

Forty symbols, 161 streams, seven calculators and thirty books read the same feed. During a burst, a depth handler that blocks must not delay a price.

The need	What ATLAS provides	How CryptAlyst delivers it	Proof of delivery
Priority must be a property of the traffic, not a promise in a comment.	Plane in metadata plus separate topology: the market plane has its own exchange, queue family and consumer budget.	CryptAlyst marks every market message plane=market, publishes it on its own exchange, and records lag per class.	Market-plane lag stays under one bar interval while the compute plane is saturated.

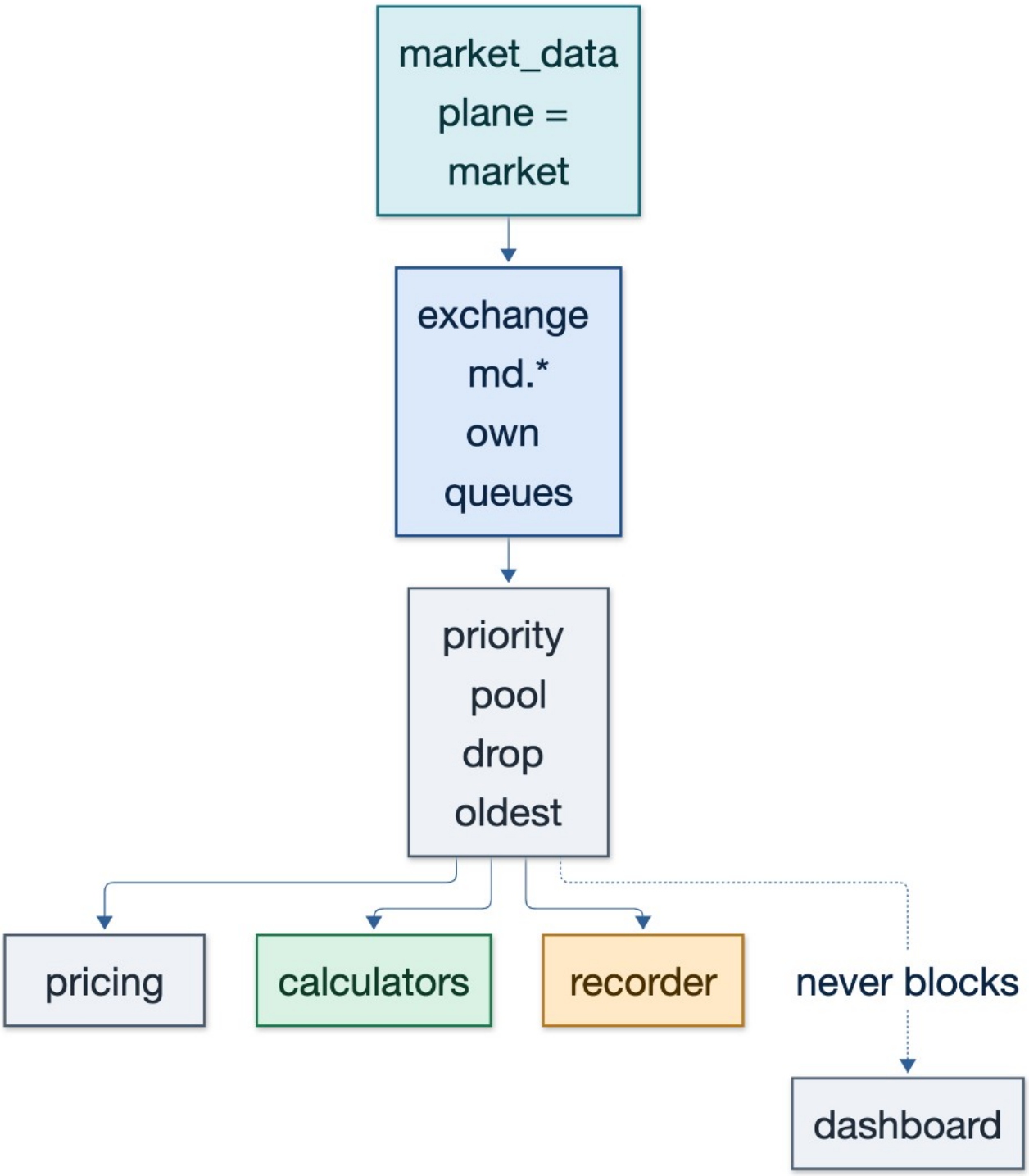


Figure — Market data stays first when compute saturates

UC-02 Compute once, read many

Three models recompute the same flow reading from the same candles, arriving at slightly different numbers because the reads happen at different moments.

The need	What ATLAS provides	How CryptAlyst delivers it	Proof of delivery
A computed indicator must be published as a value with provenance, not recomputed per consumer.	Advisory envelopes: value, horizon, confidence, expiry, and references to the inputs used.	Calculators publish intel.* advisories; engines subscribe and read rather than re-derive.	Two consumers of the same advisory return identical values and identical input references.

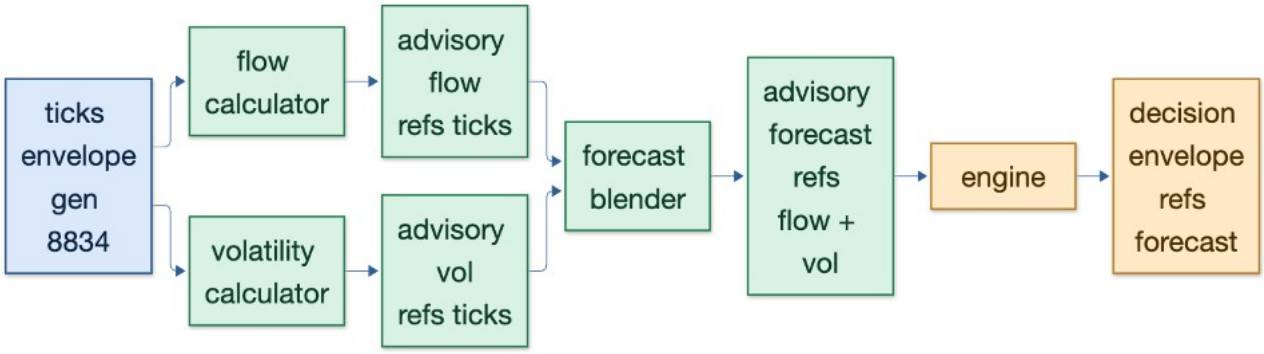


Figure — Compute once, read many

UC-03 Why did we buy?

A reviewer asks why the platform bought BTCUSDT at 09:41. Today the answer is a log line written by the model about itself.

The need	What ATLAS provides	How CryptAlyst delivers it	Proof of delivery
A decision must resolve to the advisories behind it and the candles behind those.	Mint locally, map centrally, resolve under authority; references accumulate as the envelope travels.	The recorder links each decision document to the advisory documents for its symbol and serves a gated trace.	One query returns decision to advisory to ticks to candle, and records that the query happened.

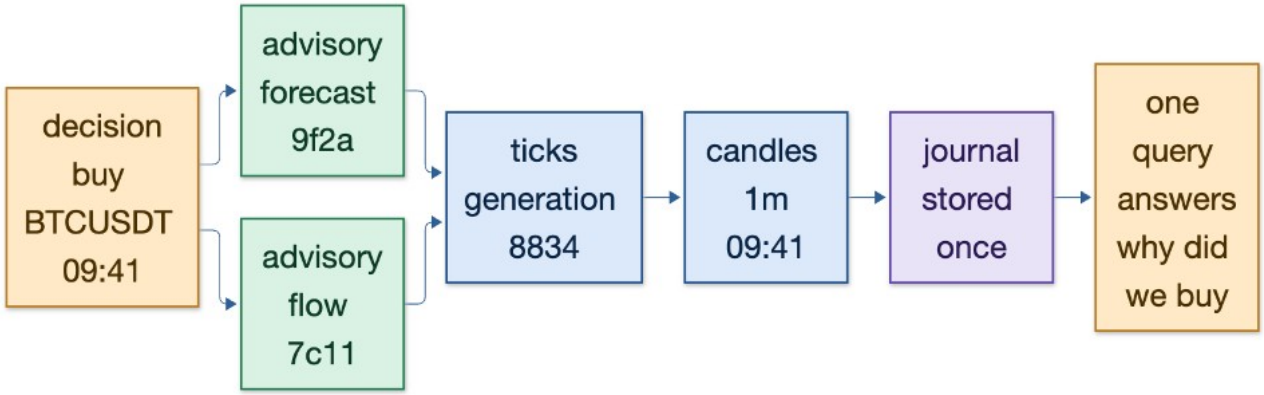


Figure — Why did we buy?

UC-04 Backtest equals live

The backtest harness reads REST candles and produces numbers that do not correspond to live trading, so research and production are different programs.

The need	What ATLAS provides	How CryptAlyst delivers it	Proof of delivery
Replay must be of the exact objects live consumers saw.	An append-only journal with replay from an offset; the stored object is the envelope.	The recorder writes every envelope to the lake; the replay tool re-emits a window to the bus at a chosen speed.	Replaying a recorded window reproduces the same decisions as the live run.

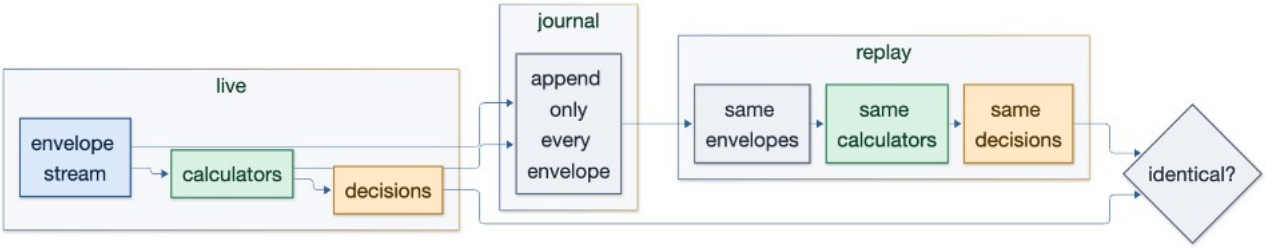


Figure — Backtest equals live

UC-05 Refusing stale input with evidence

A forecast older than its limit is refused, but the refusal quotes a wall-clock guess rather than the envelope that produced it.

The need	What ATLAS provides	How CryptAlyst delivers it	Proof of delivery
Freshness must be carried by the object, and a refusal must quote it.	computed_at and generation in metadata; the age is a property of the envelope.	Every advisory carries computed_at and generation; the risk gate refuses on age and records the numbers.	A refused decision names the age, the limit and the generation it came from.

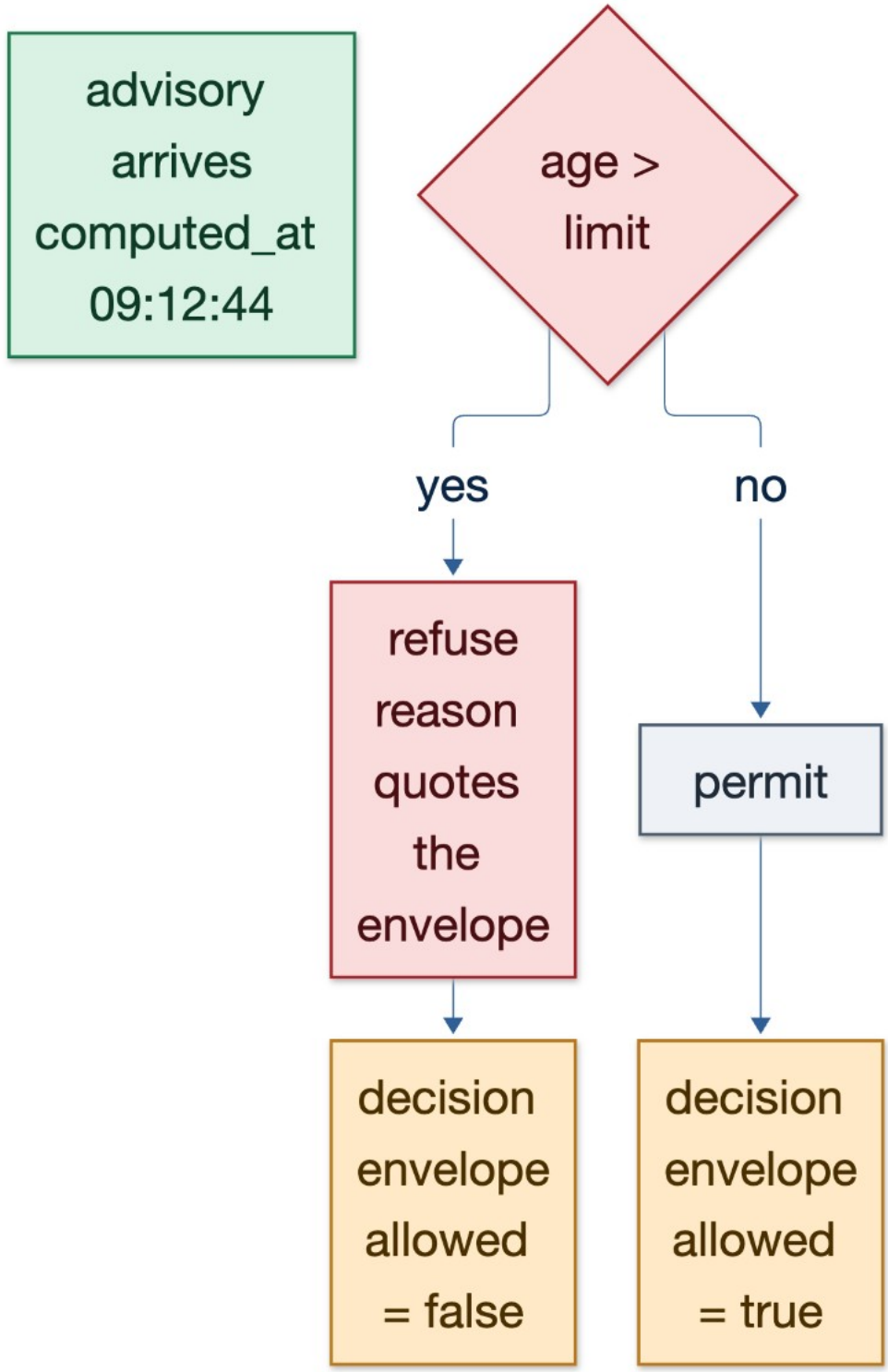


Figure — Refusing stale input with evidence

UC-06 A new model is a registration

Adding a model currently means editing a supervisor table, an engine list and a dashboard map.

The need	What ATLAS provides	How CryptAlyst delivers it	Proof of delivery
New capability must not reopen existing code.	Operators advertise capabilities; callers ask for the capability, not the service.	A model registers a capability name and its subscriptions; nothing else changes.	A new model trades after a registration with no diff outside its own directory.

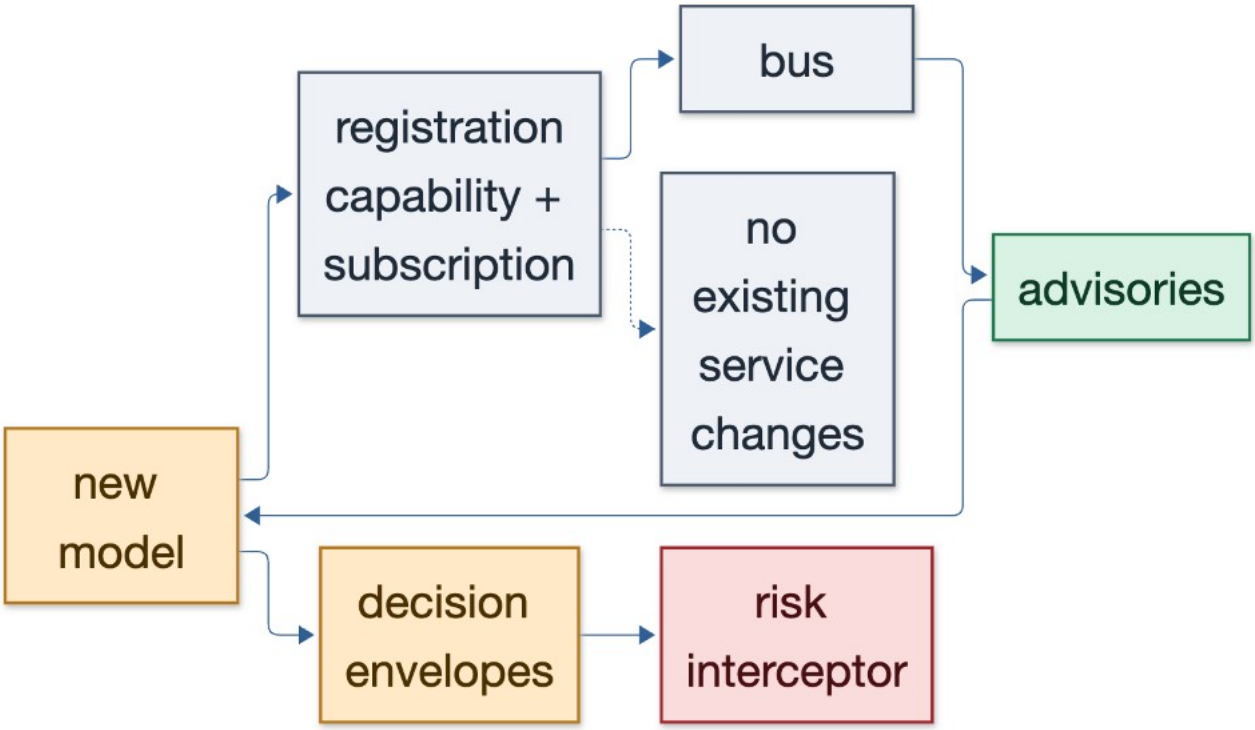


Figure — A new model is a registration

UC-07 A refusal is a first-class record

Risk refusals are visible on a dashboard but not stored with the numbers that caused them, so they cannot be reviewed later.

The need	What ATLAS provides	How CryptAlyst delivers it	Proof of delivery
Authority runs before the action and produces evidence as an output.	Interceptors refuse with the rule named; the refusal is a step on the document.	Every refusal is emitted as a decision envelope with allowed=false and the rule quoted.	Every refusal on the dashboard can be retrieved later with its inputs.

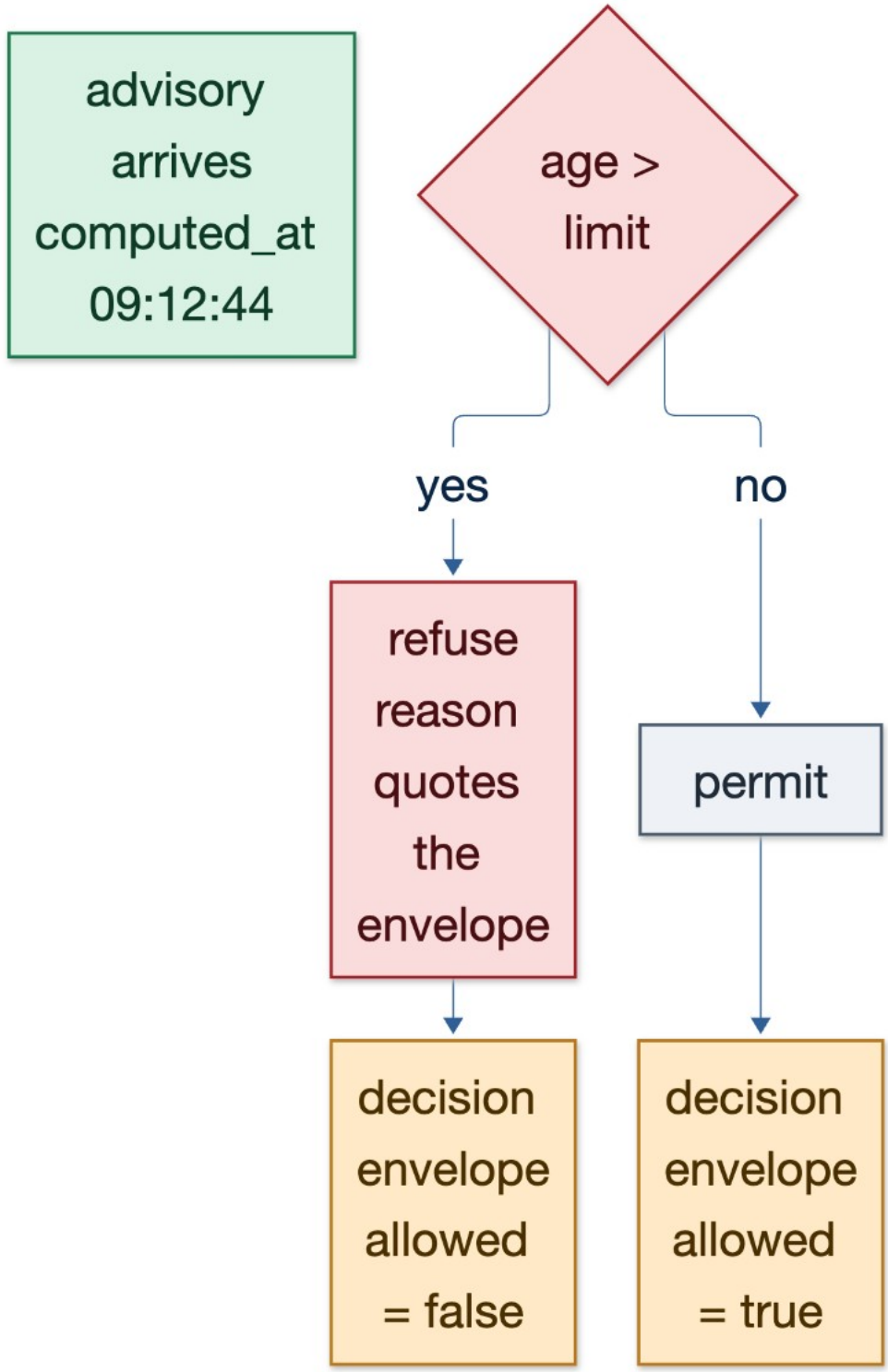


Figure — A refusal is a first-class record

UC-08 Storage growth with a policy instead of a surprise

The lake recorded 272 MB in fifteen minutes on the first run — approximately eighteen gigabytes a day — almost all of it one oversized snapshot.

The need	What ATLAS provides	How CryptAlyst delivers it	Proof of delivery
Recording must be affordable, which means retention and payload discipline are part of the design.	Tiered storage with rotation, compression and expiry; a payload is the smallest object that answers the question.	The snapshot now carries only streamed symbols; segments rotate, compress at 44.7% and expire by tier.	Lake growth has a stated window per tier and the journal never grows unbounded.

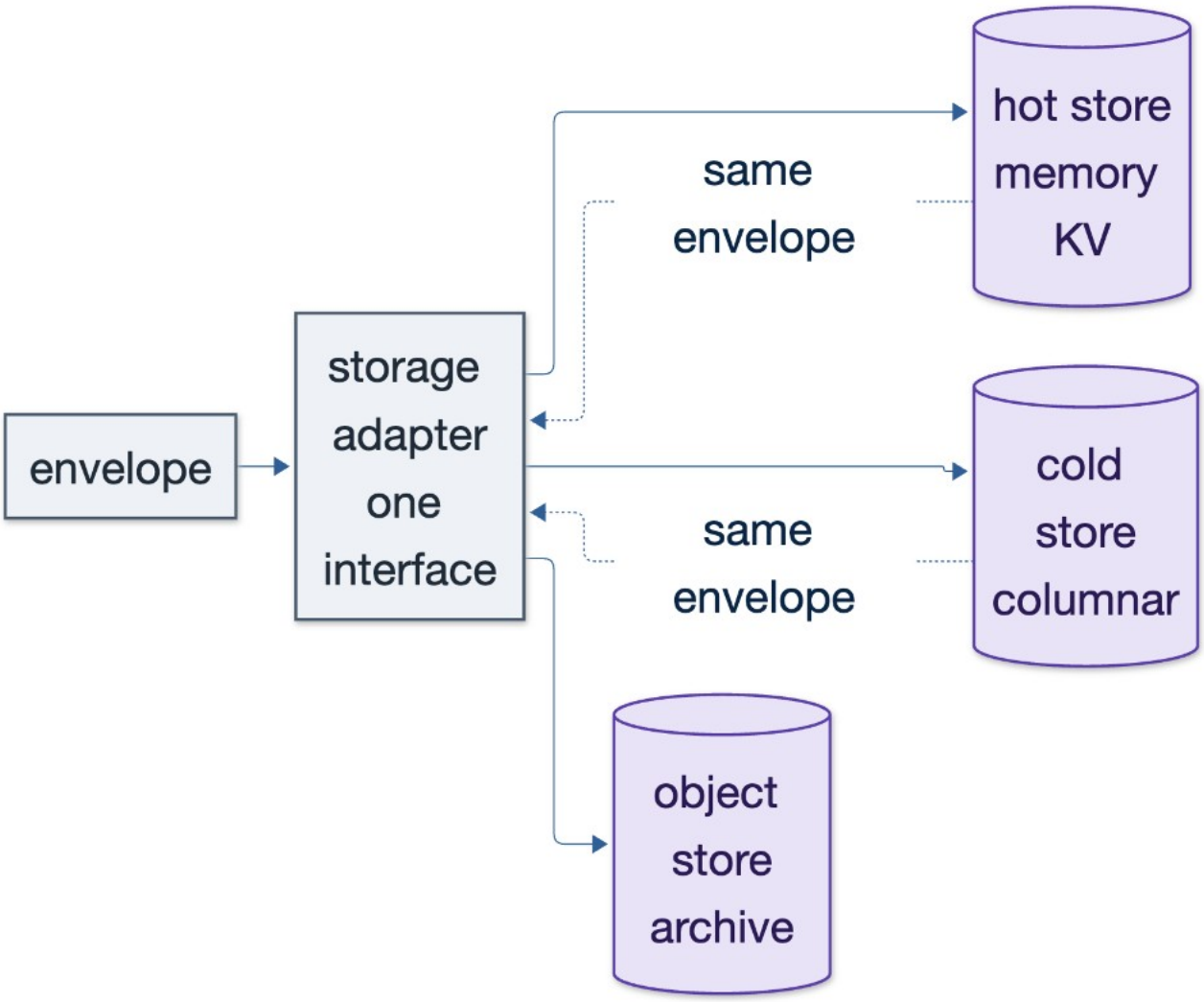


Figure — Storage growth with a policy instead of a surprise

UC-09 The dashboard explains rather than displays

The dashboard shows numbers. A number without provenance cannot be acted on with confidence.

The need	What ATLAS provides	How CryptAlyst delivers it	Proof of delivery
Every visible value should be able to say when it was computed and from what.	Metadata travels with data, so the UI reads age and generation rather than guessing.	Panels show computed_at and generation; the model drill-down shows the envelope chain behind a decision.	Every panel that shows a derived value can show its age and producer.

UC-10 Incident forensics at 09:41

Something went wrong at a specific minute. Reconstructing it means reading four log files and hoping they share timestamps.

The need	What ATLAS provides	How CryptAlyst delivers it	Proof of delivery
The platform must be able to replay a window and explain each decision in it.	Correlation map plus journal: replay the window, then resolve the documents it touches.	Replay the minute, resolve each decision, compare against the live record.	The reconstruction identifies the advisory and the rule responsible for each decision in the window.

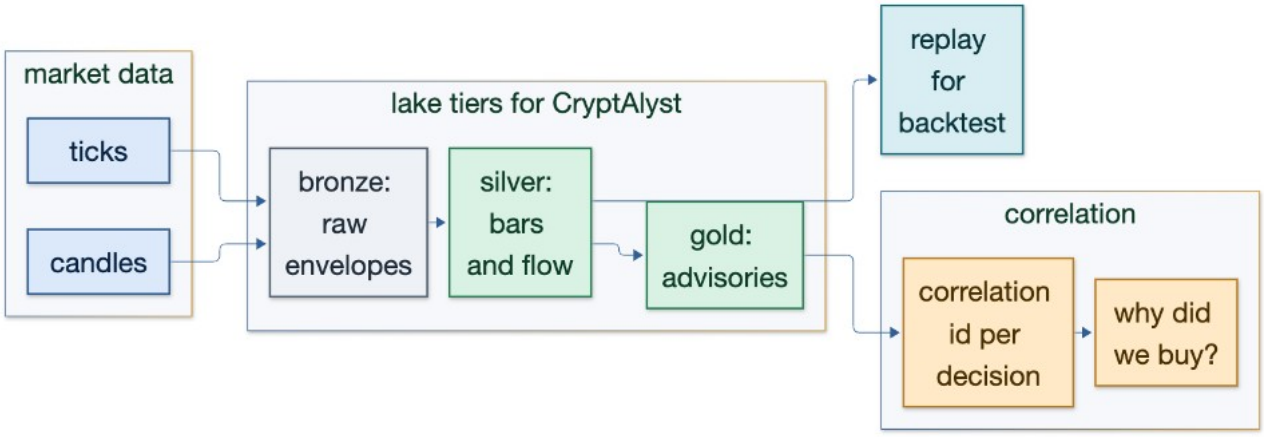


Figure — Incident forensics at 09:41

UC-11 Model decay without a separate research pipeline

A model that worked in walk-forward stops working live, and there is no shared dataset connecting its live decisions to its backtest.

The need	What ATLAS provides	How CryptAlyst delivers it	Proof of delivery
The dataset used for research must be the dataset produced in production.	One lake of envelopes feeding both replay and study; gold tier holds per-model aggregates.	Per-model performance is materialised from decision envelopes, so decay is measured on the same objects.	A decay curve can be computed from gold without touching the live system.

UC-12 Compliance evidence produced by doing the work

An auditor asks what the platform did, why, and who looked at it. Today this is an exercise in log archaeology.

The need	What ATLAS provides	How CryptAlyst delivers it	Proof of delivery
Evidence must be an output of the work, not a report written afterwards.	Steps on documents, gated resolution, and the read itself recorded as a step.	Decision envelopes carry the rule, the inputs and the actor; resolution records the reader.	An evidence pack is a query over the correlation map and the lake, not a reconstruction.

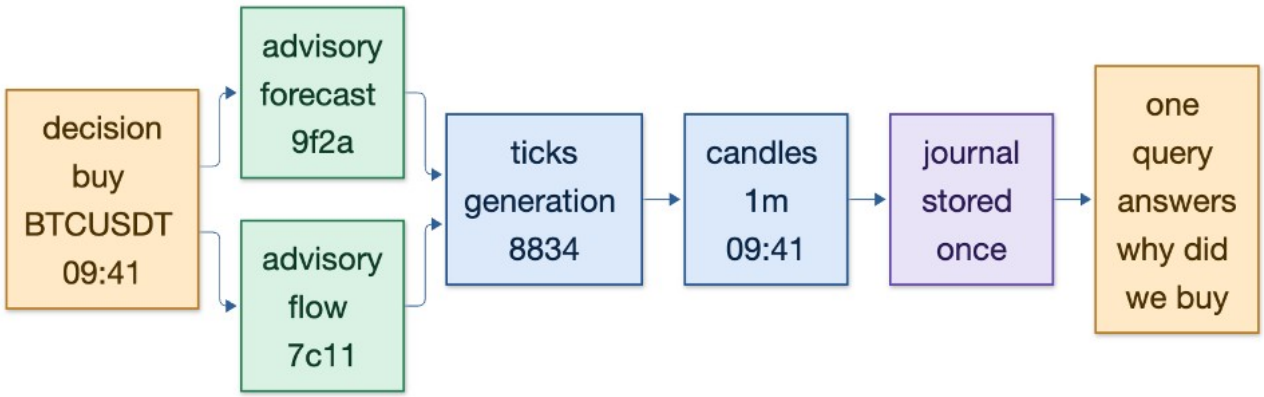


Figure — Compliance evidence produced by doing the work

UC-13 Thirty books without a dependency tangle

Thirty models share four inputs; today that graph exists only in the code that fetches them.

The need	What ATLAS provides	How CryptAlyst delivers it	Proof of delivery
Dependencies must be declared, inspectable and stable as books are added.	Subscriptions are the graph; advisories are the shared edges.	Each book declares its topics; the bus renders the graph and the dashboard can show it.	Adding or removing a book changes the graph, not the code of any other book.

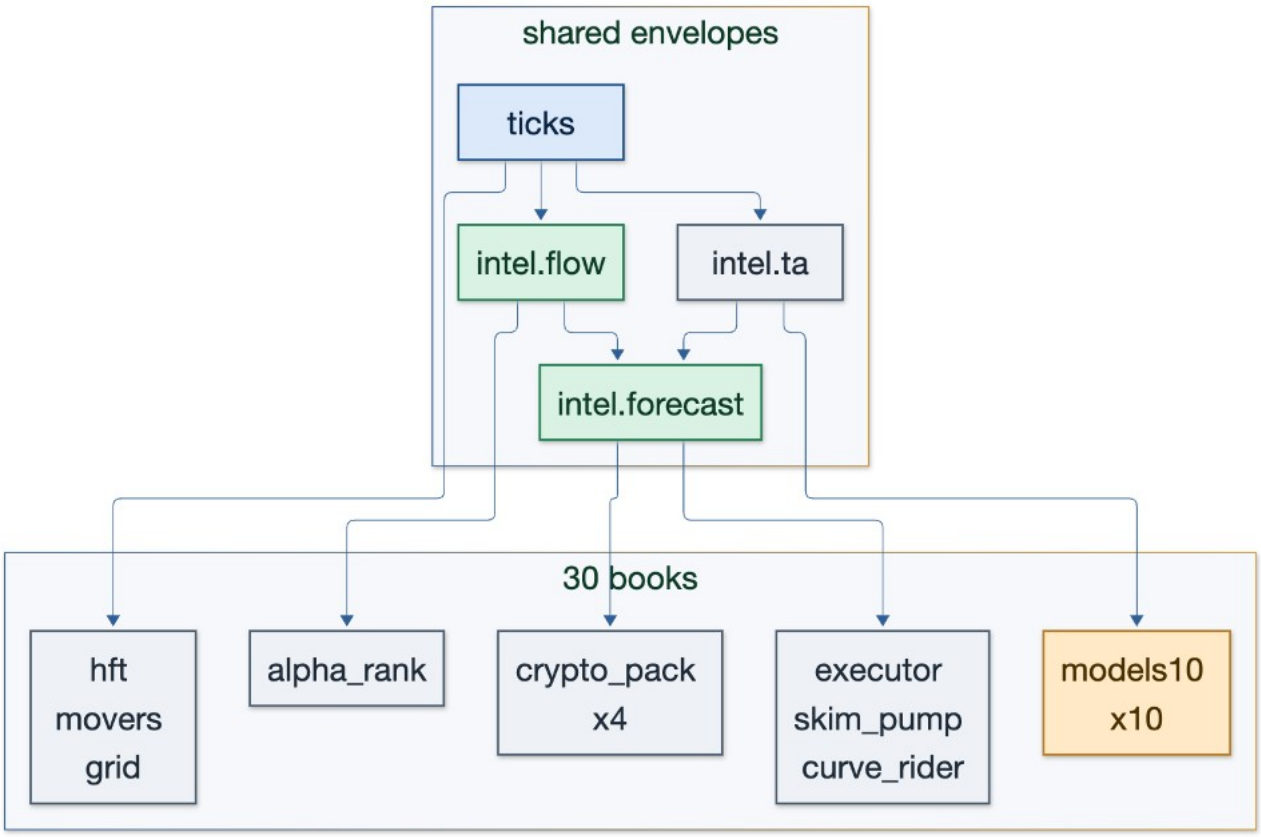


Figure — Thirty books without a dependency tangle

UC-14 Research without touching production

A researcher wants a week of history without loading the live broker or the live lake's write path.

The need	What ATLAS provides	How CryptAlyst delivers it	Proof of delivery
History must be readable in isolation, at any speed.	Replay from a copied journal into a separate topic namespace.	Copy segments, replay into a research instance of the calculators, compare outputs offline.	Research runs read-only, from cold segments, with no impact on live lag.

UC-15 A failed producer is a gap, not a mystery

A producer stops for ten minutes. Consumers cannot tell whether the market was quiet or the feed died.

The need	What ATLAS provides	How CryptAlyst delivers it	Proof of delivery
Absence must be observable, and recovery must not require re-derivation.	generation and sequence in metadata; the recorder records what arrived and the orchestrator notices what did not.	The recorder exposes per-topic receive rate and last-seen time; the orchestrator alarms on a generation gap.	A producer outage is visible as a gap in generation, not as a quiet dashboard.

UC-16 Cost accounting per model

The platform knows total fees but not which model spends them, so a model can be net-negative and invisible.

The need	What ATLAS provides	How CryptAlyst delivers it	Proof of delivery
Cost must be attributable to the decision that incurred it.	Decision envelopes carry notional, fee assumptions and the model identity.	Fills are recorded with their decision reference, so attributed cost is a gold-tier aggregate.	Each model reports gross edge, fees and net in one query, with the trades behind it.

Part IV — Delivery

13 Measured performance

Every number below was produced on this machine against the running platform. Nothing here is a target.

Measurement	Result	How it was obtained
envelope decode	2.46 µs per envelope, 978 B, 12 allocations	Go benchmark, 5,000 iterations
envelope build and encode	1.08 µs per envelope, 912 B	Go benchmark
memory write	1.85 µs per write	Go benchmark
journal write	2.39 µs per write	Go benchmark
lake fan-out (memory and journal)	2.41 µs per write	Go benchmark
end-to-end write	200,000 envelopes in 500 ms = 400,000 per second	timed test
end-to-end replay	200,000 envelopes in 177 ms = 1,127,000 per second	timed test
live recorder throughput	27.8 per second sustained, 0 errors	running service, :8801
ticks envelope	7.4 KB after the snapshot fix, was 98.6 KB	lake inspection
depth publish rate	38.8 per second across 40 symbols	broker statistics
compression on live traffic	44.7% of original size	gzip of the live journal
test suite	21 tests across four packages, all passing	go test ./...

13.1 What the numbers mean

The envelope and the lake are three orders of magnitude faster than the traffic they carry. The platform publishes about 28 messages per second and the recorder can absorb 400,000. Therefore neither the envelope nor the lake is a performance risk, and any latency the platform has is in the models, not in the carrier.

The number that did matter was payload size, not rate: a snapshot carrying the full mini-ticker universe made each ticks envelope 98.6 KB and turned into eighteen gigabytes a day. Restricting the snapshot to streamed symbols reduced it by a factor of thirteen and is covered by a unit test so it cannot regress.

14 Test and verification strategy

14.1 What is tested

Package	Tests	What they protect
internal/envelope	7	round trip, unknown member refused, unknown meta key ignored, version policy, refs accumulate, age, transformers
internal/correlation	5	cid uniqueness over 1,000 mints, append, gated read recorded, ancestry ordered root-first, cycle safety
internal/lake	6	memory keeps latest and drops oldest, journal round trip, replay across segments, fan-out, rotation, compression, expiry
internal/marketdata	3	snapshot restricted to streamed symbols, depth change detection, candle series bounded

14.2 What is verified against the running system

- The broker, not the dashboard: message counts and rates are read from the management API.
- The lake on disk: record counts, topic mix, and bytes per topic are measured from the journal itself.
- The recorder's own counters: received, written, errors, rate, correlation documents, steps and reads.
- A trace query end to end: a decision document resolves with its steps recorded.

Three real defects were found by this strategy and are worth recording, because they are the class of bug the design exists to prevent: an ancestry that came back leaf-first (fixed by reversing the walk), a benchmark that was accidentally O(iterations x records) and hung the suite, and a JSON field collision where a case-insensitive decoder matched a numeric exchange field to a string field and silently discarded every candle.

15 Rollout plan and acceptance gates

Each phase leaves the platform running. A phase is complete when its gate is measured, not when the code is merged.

Phase	Change	Gate
1	Envelope in Go; the four Go services wrapped at the boundary	every recorded message decodes with version, plane and producer
2	Recorder with tiers, rotation, compression and expiry	a window replays exactly; lake growth is bounded by policy
3	Calculator tier publishes advisories with references	two consumers of one advisory see identical values and refs
4	Orchestrator: capabilities, deadlines, generation checks	a stale advisory is superseded, not recomputed by every consumer
5	Books emit decisions with cid and references	a decision resolves to advisory, ticks and candle in one query
6	Replay wired to the backtest harness	live and replay agree on the same recorded window
7	Retention tuned per tier; archive verified	storage stays within its stated window for a week of running

15.1 The one invariant

The envelope never changes shape. Everything that varies — venues, indicators, models, stores — lives in operators, interceptors and transformers around it. The moment the envelope grows a trading-specific field, the design has failed and the platform is back to a bespoke message per consumer.

16 Risks, open decisions and the invariant

16.1 Risks

Risk	Likelihood	Mitigation
Python books never emit envelopes	medium	phase 5 is small: wrap the existing decision payload and add cid and refs
Storage outgrows the disk	high without policy	tiered retention, compression, and per-topic payload budgets
Schema drift between producers	medium	refuse unknown top-level members; version the payload before there are two producers
Orchestrator becomes a second scheduler	medium	capabilities only: it may ask for a name, never a process
Correlation becomes a single point of failure	low	minting is local; the map is a rebuildable projection

16.2 Open decisions

- Retention duration per tier — needs a stated research horizon before it can be fixed.
- Whether advisories are attributed to the model that computed them or to a separate calculator tier.
- Whether depth is published as five levels on change or as a cheaper top-of-book every second.
- Whether the archive is local disk or object storage, which decides the cost of the compliance use case.

16.3 Closing statement

This edition replaces opinion with lineage. The platform can now say what a message is, when it was computed, what it was computed from, and who read it. That is not a cosmetic improvement: it is the difference between a system that accumulates trades and one that can learn from them.

Part V — Storage, security and the ticker service

18 Storage selection and sizing

The lake defines the shape of storage: tiers behind one interface. This chapter chooses the actual products, and — more importantly — states the measurement that would force each choice to change. A recommendation without a switch trigger ages into advocacy.

The constraint that decides most of this: one operator, one machine, no container runtime. Every technology below installs as a single binary or a Python package, and none of them requires a second person to run.

18.1 The recommended stack

Data	Store	Why this one	Switch trigger
Operational state — books, positions, orders, fills, risk decisions	SQLite (WAL), one file per book	ACID, zero operations, already present on the machine; per-book files mean no write contention	two writers on one table, or remote concurrent access → Postgres
Bronze, silver and gold envelopes	Parquet files, partitioned by topic and date, queried with DuckDB	immutable files, zstd compression, full SQL with no server to run	gold beyond 10^9 rows, or concurrent analytical users → ClickHouse
Correlation map — documents, steps, references, cids	SQLite with three tables and indexes on cid and doc_id	a ledger with lookups, not a document store; rebuildable by replay, so a lost file is cheap	multi-instance resolution or multi-user gating → Postgres
Hot reads — latest prices, book snapshots, dashboard	Go process memory	40 symbols of small structs; a network hop would be slower and less reliable	more than one dashboard instance → Redis as a cache, never as truth
Archive of closed segments	local disk with gzip now; MinIO when off-box retention is required	compression already measured at 44.7% of original size	retention beyond one machine, or an auditor who needs independent access
Bus	RabbitMQ (keep)	already the spine; one bus is an architectural rule, not a preference	replay semantics outgrow queues → NATS JetStream (single binary), not Kafka

18.2 Sizing arithmetic

```
live rate            28 msg/s            = 2.4M envelopes/day
payload              ticks 7.4 KB, depth 0.4 KB, advisories 0.1 KB
raw bronze           ~18 GB/day
after gzip (44.7%)   ~8 GB/day
after parquet+zstd    ~1.5-2 GB/day
SQLite comfortable   to ~50 GB per file
DuckDB scan          2 GB parquet in tens of milliseconds
```

These are the numbers that decide when to move. Today's traffic is two orders of magnitude below what Parquet and DuckDB handle comfortably, and three below ClickHouse. Adopting ClickHouse now would be buying headroom with operational cost.

18.3 What not to use, and why

Rejected	Why
Kafka	operationally heavy for one operator, and its log duplicates what the recorder already provides
MongoDB	no cross-document transactions; the correlation ledger needs atomic step appends alongside document state
Elasticsearch	wrong cost curve for tick data; 18 GB/day indexed to answer questions a Parquet scan answers better
TimescaleDB	adds a server for a downsampling job that partitioned Parquet already does
Redis as truth	the moment book state lives only in Redis, a restart is data loss

Docker as a prerequisite	a paper-trading platform should not need a container runtime to book a trade

18.4 Backup and recovery

Store	Backup	Recovery
SQLite	online `.backup` per file, nightly	copy the file back; WAL replays
Parquet	immutable; copy segments to a second disk	files are the record; no restore step
RabbitMQ	export definitions and bindings	redeclare; consumers reconnect automatically
Lake journal	gzip segments on rotation	replay from any segment boundary

19 The security service

Security in this platform is not a login page. It is the component that answers three questions before any work happens: who is asking, on whose authority, and what may they see of the answer. It is built in the ATLAS style — an interceptor before the operator, a gate on resolution, and a ledger entry for every decision including the refusals.

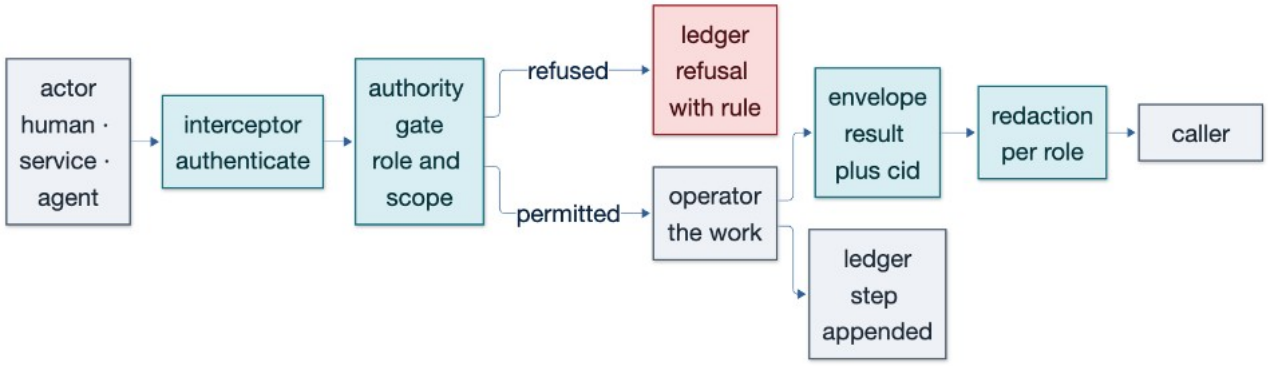


Figure 19.1 — Authenticate, gate, do the work, redact for the caller, and write the step.

19.1 The three parts

Part	Responsibility	Failure it prevents
Interceptor	establish the actor: a dashboard user, a service, or an agent	anonymous work with no attributable actor
Authority gate	decide whether this actor may do this, with the rule named	unreviewable access to positions, traces or keys
Redaction	shape the answer for the caller's role without changing its truth	one role leaking another role's data

19.2 Roles and what each may see

Role	Sees	Never sees
Owner	its own documents in full, with references	other owners' payloads
Peer service	the needs it may serve, its own steps, artifact names	another service's pricing inputs
Auditor	actors, outcomes, timings, requirements, artifacts	payloads, only redacted references
Bystander	state and requirement satisfaction only	any step detail

Every resolution is recorded as a `read` step on the document being read. That single rule answers the question a desk asks first in an incident: who looked at this, and when.

19.3 Credentials and keys

- Service to service: short-lived introduction tokens minted by the correlation service, not shared secrets in config files.
- Human: session token on the dashboard, scoped to a role, with the read recorded.
- Exchange keys: never on the bus, never in an envelope; held by the execution adapter only, referenced by name.
- Every refusal is a step with the rule quoted, so an attempted escalation is visible rather than silent.

The gate is not optional. A trace reveals who asked what, about which symbol, at what time — the platform's memory of intent. Shipping a resolver before its authorization converts a transient packet into a queryable record of intent with nothing protecting it.

20 The ticker service

The ticker service is the public face of the market plane: a price feed for dashboards, external clients and any service that should not open a bus connection. It is a thin operator over the market plane — it computes nothing, caches what already exists, and answers fast.

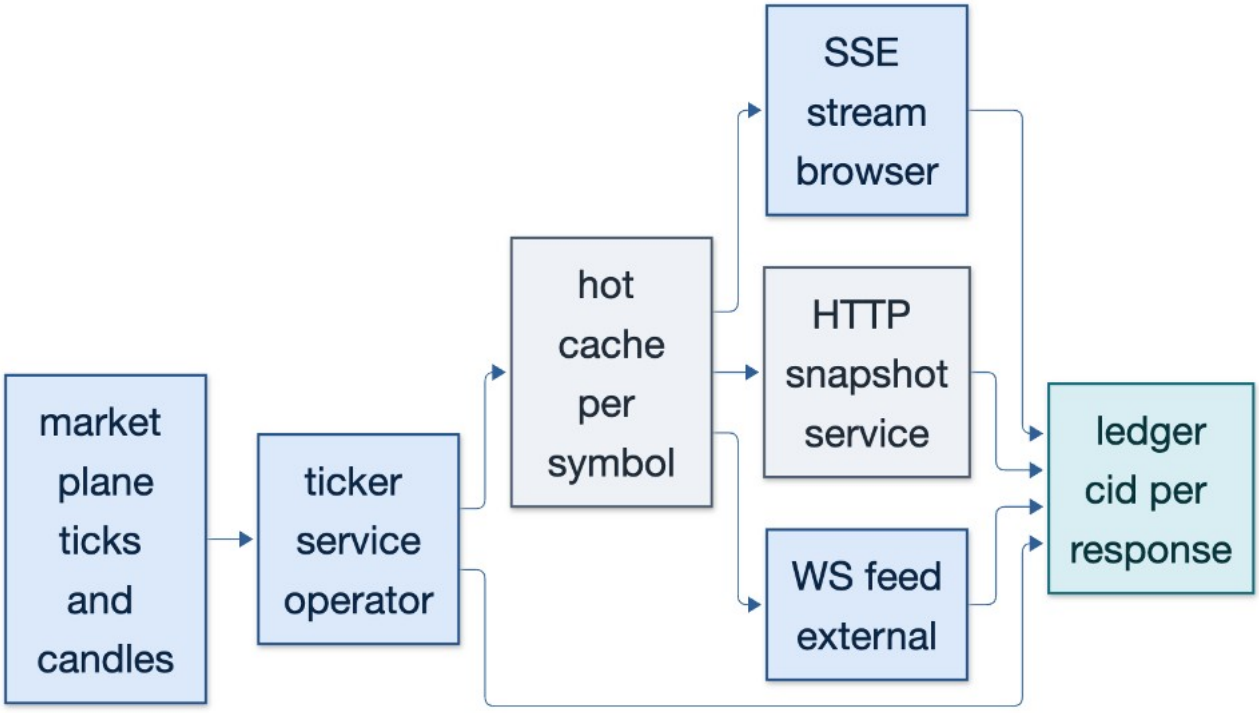


Figure 20.1 — One operator, a hot cache, three delivery shapes, and a ledger entry per response.

20.1 Responsibilities

Responsibility	Behaviour	Why
Subscribe	reads the market plane once; never polls the exchange	the feed stays the only process that talks to Binance
Cache	keeps the latest tick, candle and 24h statistic per symbol in memory	answers in microseconds without touching the bus per request
Serve	SSE stream for browsers, HTTP snapshot for services, WebSocket for external clients	one source, three shapes, no duplicated logic
Trace	mints a cid per response and records what was served	a client can quote the id and the platform can resolve it
Gate	redacts per role; a bystander sees prices but not book depth	public data is not the same as proprietary data

20.2 The envelope it returns

```
envelope(ticker)
  data:  { symbol: BTCUSDT, last: 82750.12, bid: 82750.11, ask: 82750.13,
           change_24h: +1.82, quote_volume: 2465058 }
  meta:  { version: 1, plane: market, producer: ticker,
           computed_at: 09:41:02.114, generation: 8834, cid: cid-9f2a41 }
  refs:  [ md.ticks@8834, md.candle@8834 ]
```

A client that receives this can quote the cid in a support request, and the platform can resolve exactly which tick and which candle produced the number it saw.

20.3 Interface

Route	Shape	Consumer
GET /ticker/{symbol}	one envelope, JSON	services and scripts
GET /ticker/stream	SSE, one event per change	dashboard

WS /ticker	frames of envelopes, one per symbol per change	external clients
GET /ticker/trace/{cid}	gated resolution of a served response	support and audit

20.4 Why this is a service and not a route

A ticker route on the dashboard would work until a second consumer appeared, at which point the platform would have two definitions of a price. Making it a service with an envelope and a cache means one definition, one trace, one place to fix a rounding rule — and the dashboard becomes just another client.

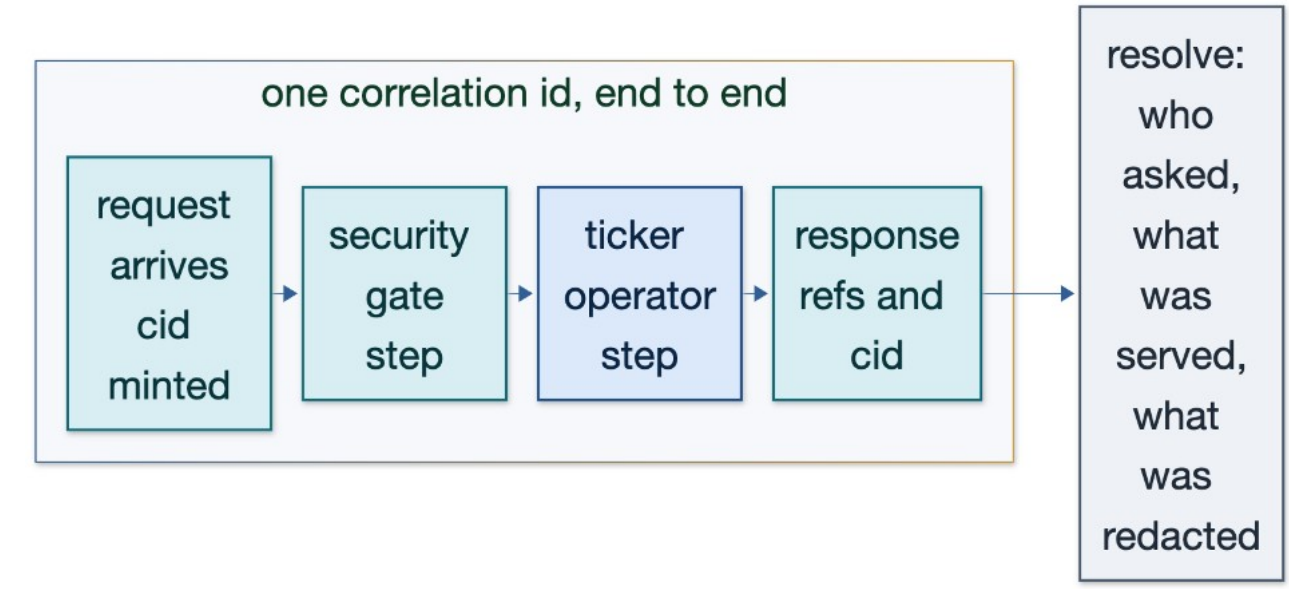


Figure 20.2 — One correlation id across security and ticker: who asked, what was served, what was redacted.

21 Appendix

21.1 Topic table

Topic	Exchange	Class	Producer
ticks, market.price, market.depth, market.candle, market.mini24h	market	market	marketdata (Go)
intel.signal, intel.flow, intel.movers	intel	intel	signals (Python)
intel.ta	intel	intel	tad (Go)
intel.forecast	intel	intel	forecast (Go)
alert.pumpdump	alert	control	signalbus (Go)
model.decision, model.fill, model.pnl, model.rank	model	model	books (Python)
risk.state, exec.quote	risk / exec	control	risk, maker
system.health, review.verdict	system / review	system	healthpub, review

21.2 Metric definitions

Metric	Definition
age	now minus computed_at, in milliseconds; the number an engine refuses on
generation	the store or compute counter an envelope was produced from
lag	publish time to consume time for a class of traffic
drop	a market-plane message discarded because a consumer fell behind
rate	messages received per second, measured by the recorder, not estimated

21.3 File map

Path	Contents
core/internal/envelope	the contract and its version policy
core/internal/correlation	the map, minting and gated resolution
core/internal/lake	backends, retention, replay
core/internal/marketdata	live state, snapshots, depth change detection
core/cmd/recorder	the persistence plane
core/cmd/replay	replay to the bus
docs/envelope	Mermaid sources and rendered diagrams

Appendix A — Database technology considerations

Chapter 18 chooses; this appendix explains the reasoning behind the choices and states what would have to be measured to overturn them. It is written to be argued with, because a storage decision that cannot be challenged on evidence is a preference rather than an engineering decision.

A.1 Access patterns by data class

Data class	Write rate	Read shape	Retention	Dominant cost
ticks and depth	tens per second	latest value, window scans	hours to days	write volume and payload size
candles	one per bar per symbol	ranges, aggregates	weeks	scan width
advisories	tens per second	latest per symbol, per-model history	weeks	cardinality of symbol and model
decisions and fills	tens per minute	exact lookup by cid, per-model aggregates	years	durability and audit
correlation steps	one per append	ancestry walk, time range	years	index size
dashboard reads	hundreds per minute	small, cached, repeated	none	latency

A.2 Candidate matrix

Technology	Model	Write	Query strength	Ops cost	Concurrency	Verdict here
SQLite (WAL)	relational, embedded	excellent single-writer	good with indexes	none	one writer, many readers	operational state and the correlation map
Postgres	relational, server	very good	excellent, JSONB and windows	low	MVCC, many writers	when concurrency or gating needs it
ClickHouse	columnar, server	excellent, batched	excellent for aggregates	medium	many writers, merge on read	gold at scale, not yet
DuckDB + Parquet	columnar, embedded	append-only files	excellent for scans	none	single process	research and bronze/silver
TimescaleDB	Postgres extension	very good	time-series functions	low	as Postgres	viable, redundant with Parquet downsampling
MongoDB	document	good	weak joins, no cross-doc transactions	medium	sharded	rejected for the ledger
Redis	in-memory KV	excellent	no complex queries	low	single-threaded per shard	cache only, never truth
Kafka	distributed log	excellent	no queries, consumer groups	high	partitions	rejected at this scale
NATS JetStream	log, single binary	excellent	replay by sequence	low	lightweight	the migration target if the bus outgrows queues
MinIO / S3	object store	good, immutable	none without an engine	medium	eventually consistent	archive tier when off-box is required

A.3 Concurrency and contention

The single-writer restriction of SQLite looks like a limitation and is, here, a design tool. Each book owns its own file, so thirty books do not contend; each file is trivially backed up, and cleaning a book is a file operation rather than a transaction against shared state. Postgres becomes necessary at exactly the moment two processes must write the same table — which in this design is the correlation service when it serves more than one reader instance, and the risk state when it is consumed by more than one writer.

```
SQLite writer model      one writer at a time, WAL allows concurrent readers
per-book isolation       30 files, 30 independent writers, no contention
```

```
contention appears when  two processes write one table, or a table exceeds ~50 GB
Postgres trigger         concurrent write requirement, not volume
```

A.4 Durability and the fsync trade

Setting	Behaviour	Cost	Use
fsync per record	every write survives power loss	thousands of writes per second	fills, decisions, risk state
group commit (batch)	writes survive in batches	hundreds of thousands per second	market data, advisories
WAL + periodic flush	small window of loss	very high throughput	bronze market data only

The rule: anything that must explain a trade is fsynced; anything that can be re-read from the feed is not. A lost tick is a gap; a lost fill is an audit failure.

A.5 Schema evolution

The envelope makes schema evolution a metadata problem rather than a migration problem. Payloads version independently of the envelope; a reader accepts the current revision and the one before it. In a columnar store this means: keep the envelope columns stable (topic, key, generation, computed_at, cid), and store the payload as a JSON column or a struct that can gain fields. Materialise a payload field into a column only when a query needs it at scan speed.

A.6 What each engine does at scale

Rows	SQLite	DuckDB + Parquet	ClickHouse
1 million	instant with an index	instant	instant
100 million	slow for scans, fine for lookups	sub-second scans	sub-second aggregates
1 billion	not appropriate	seconds per scan, parallel	sub-second with partitioning

A.7 Migration path

From	To	Trigger	Method
SQLite	Postgres	concurrent writers, or remote access	dual-write one week, then read-cut, then write-cut
Parquet	ClickHouse	gold beyond 10^9 rows or concurrent BI	load Parquet into a ClickHouse table; queries run in both during cut-over
RabbitMQ	NATS JetStream	replay semantics outgrow queues	publish both, compare consumer lag, cut consumers topic by topic

A.8 Benchmarks to run before deciding

- Insert 1M envelopes with fsync per record: measure records per second and p99 latency.
- Scan a day of Parquet with DuckDB for one symbol's flow history: measure cold and warm.
- Aggregate per-model net over 30 days in ClickHouse: measure with and without a projection.
- Resolve a decision chain of depth 6 in SQLite and in Postgres: measure p99 under 50 concurrent readers.
- Replay 24 hours of envelopes: measure whether the target rate is reached without drops.

Appendix B — Compliance

Compliance is treated here as a consequence of the architecture rather than a reporting layer bolted on at the end. If the platform records what explains each decision, and records who read it, the evidence pack is a query — not a reconstruction exercise.

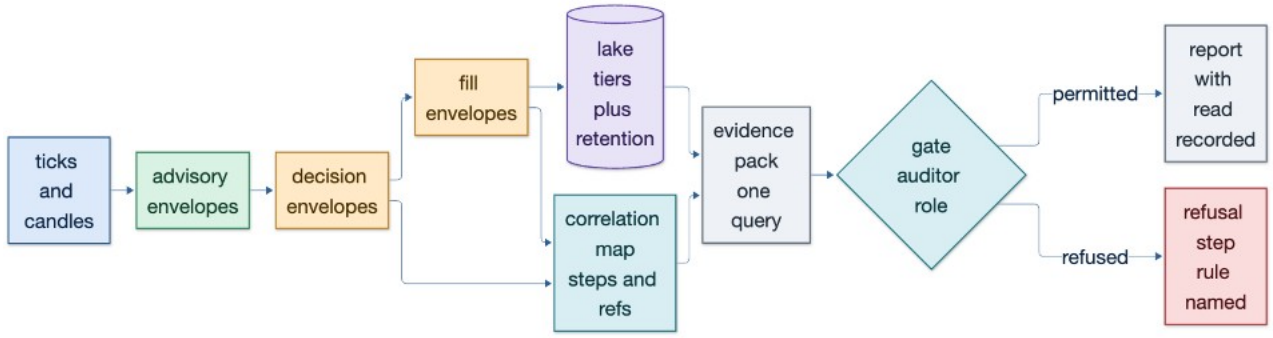


Figure B.1 — Decision and fill envelopes, the correlation map, the lake, and a gated evidence pack.

B.1 What must be evidenced

Obligation	Evidence	Where it lives
What was traded, when, at what price	fill envelopes with timestamps and notional	lake, gold tier
Why the decision was made	decision envelope with refs to advisories and ticks	lake plus correlation map
Who or what decided	producer identity and model version in metadata	envelope meta
What was refused and on which rule	decision envelope with allowed=false and the rule quoted	lake
Who looked at the record	read step appended on resolution	correlation map
That the record was not altered	hash-chained segments, append-only journal	archive
That data was not missing	generation and sequence continuity, gap detection	recorder statistics

B.2 Retention schedule

Data	Minimum retention	Rationale
ticks and depth	7 days	replay for forensic and regression windows
candles and advisories	90 days	model study and decay measurement
decisions and fills	7 years	regulatory and dispute horizon
risk refusals and gate decisions	7 years	proof of control
access and resolution logs	7 years	proof of who saw what

B.3 The evidence pack

One request produces a pack for a window: every decision, its chain, the fills it caused, the rules it passed or failed, and the identities that read it. The pack is generated by query, gated by role, and the generation is itself recorded as a step — so the act of producing evidence is auditable.

```
window        2026-10-19 09:00 to 10:00
decisions     all model.decision envelopes in window
chains        resolve each decision to advisory, ticks, candle
fills         all model.fill envelopes referencing those decisions
refusals      decision envelopes with allowed = false
readers       read steps on every document in the pack
integrity     segment hashes and continuity of generation
```

B.4 Integrity without a blockchain

Each closed segment carries a hash of the previous segment's hash, forming a chain. Verification recomputes the chain; a single altered byte breaks it at a known point. This is cheaper than a distributed ledger and sufficient for the obligation, which is to show that the record was not quietly rewritten.

```
seg-0001  hash = H(payload)
seg-0002  hash = H(H(seg-0001) || payload)
verify   -> recompute, report first mismatch by segment and offset
```

B.5 Privacy and the erasure conflict

Retention law and erasure law pull in opposite directions. The resolution adopted here: personal data is pseudonymised at the boundary (an actor id, never a name), the mapping lives in one access-controlled table, and erasure removes the mapping while the trading record keeps its pseudonym. The trade remains explainable; the person becomes unidentifiable. The alternative — deleting the trade — destroys the record the regulator requires and the evidence the desk needs.

B.6 Controls mapped to mechanism

Control	Mechanism	Test
Segregation of duties	roles in the security service; owner cannot self-audit	an owner read is refused and recorded
Access logging	read step on every resolution	every resolve appears in the map
Change control	code version in envelope metadata	replay identifies the version that produced a decision
Exception reporting	refusals, gaps and divergence as first-class records	daily report lists counts, not prose
Attestation	daily hash of the day's segments	verification script reproduces the hash

B.7 What an auditor finds when something is missing

A gap in generation is visible as a gap, not as a quiet period. The recorder exposes per-topic receive rate and last-seen time; a producer that stops leaves a discontinuity that the daily report names. That is the difference between an absence and a mystery.

Appendix C — Trade replay

Replay is the mechanism that makes a backtest a test rather than a hypothesis. It has three uses in this platform — forensic reconstruction, regression testing, and backtesting — and they share one requirement: the same inputs through the same code must produce the same decisions.

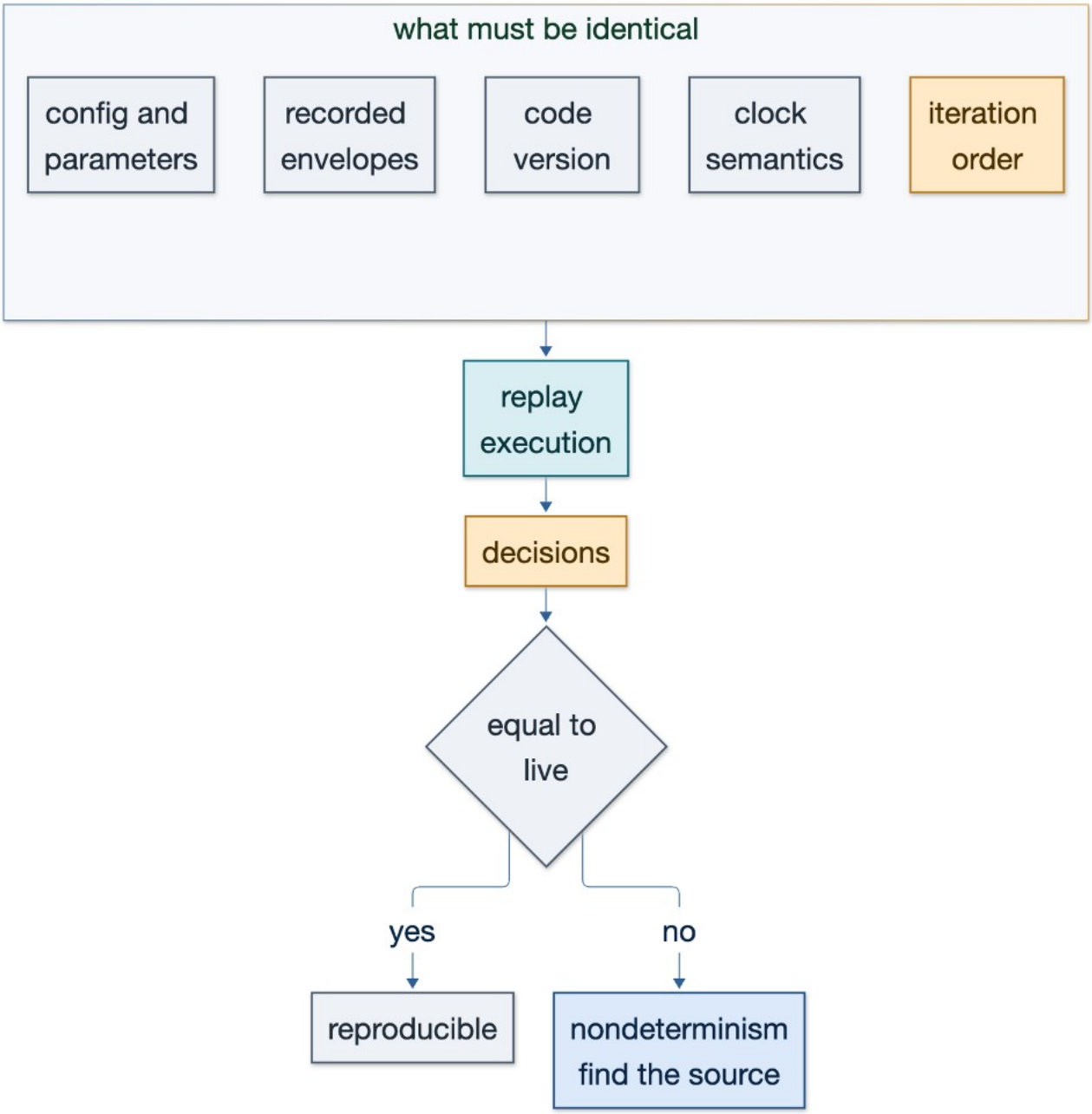


Figure C.1 — Determinism has five inputs. Any difference in one of them is a different experiment.

C.1 Three modes

Mode	Input	Purpose	Success
Forensic	a recorded window in production	explain what happened at 09:41	the chain resolves and the decisions match
Regression	a fixed recorded window in CI	prove a change did not alter behaviour	zero decision divergence, or a reviewed diff
Backtest	a long recorded window	measure a strategy over history	metrics reproducible from the same file

C.2 The five inputs that must be identical

Input	Why it matters	How it is held fixed
Envelopes	the data itself	the lake file is the input, not a re-fetch
Code version	a changed threshold is a different strategy	version in envelope metadata and in the replay manifest
Parameters	same code, different config is a different run	config hash recorded with the replay
Clock	age checks and timeouts depend on it	injected clock from envelope timestamps, never wall clock
Iteration order	maps and sets reorder results	sorted symbols, stable ordering in aggregations

C.3 How a replay runs

```
select    window from lake: topic in (...) and ts between t0 and t1
load      the exact envelopes, in original order
inject    clock = envelope.computed_at; disable wall clock and randomness
feed      envelopes through the same calculators and risk gates
capture   decisions and refusals with the same envelope shape
diff      replayed decisions against live decisions, decision by decision
report    zero divergence, or a diff naming the first point of difference
```

C.4 Divergence and what it means

Divergence	Usual cause	Action
different decision set	code or parameter change	expected if reviewed; otherwise a regression
same decision, different size	risk state differs at that moment	replay must reconstruct risk state, not read live
same decision, different timestamp	clock not injected	fix the replay harness, not the strategy
same decision, different fill price	book state not reconstructed	replay depth alongside ticks, or accept modelling tolerance
flapping between runs	map iteration order	sort keys; rerun twice to confirm determinism

C.5 Side effects are disabled, not hoped away

A replay must never place an order, write a live fill, or advance live risk state. The replay process runs with an explicit paper flag, writes to a separate book namespace, and refuses to start if the live broker connection is writable. This is a guard, not a convention: a replay that can trade is a replay nobody will run on a Friday.

C.6 Acceptance criteria

Criterion	Threshold
decisions reproduced	100% of decisions in the window appear in the replay
decision divergence	zero, or every difference reviewed and attributed
refusals reproduced	same refusal reasons and the same ages
determinism	two consecutive replays of the same window are byte-identical
replay rate	at least 10x real time, so a day replays in under three hours

C.7 Why this closes the loop

With replay, three questions stop being arguments. What happened: resolve the chain. Why: read the refs. What would have happened: replay the window. The platform no longer needs to believe its own backtests; it can compare them against the record of what it actually did, on the same data, through the same code, with the only difference being the clock.